DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Design and Evaluation of Multi-Agentic AI Systems for Contract Lifecycle Management SaaS Platforms

Nguyen, Duc Trung, B.Sc.

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Design and Evaluation of Multi-Agentic AI Systems for Contract Lifecycle Management SaaS Platforms

Entwurf und Evaluation multiagentenbasierter KI-Systeme für Contract-Lifecycle-Management-SaaS-Plattformen

Author: Nguyen, Duc Trung, B.Sc.
Supervisor: Prof. Dr. Ingo Weber
Advisor: Samed Bayer, M.Sc.
Submission Date: 04.05.2026

I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 04.05.2026 Nguyen, Duc Trung, B.Sc.

Acknowledgments

Abstract

Contract Lifecycle Management platforms increasingly rely on large language models to automate clause extraction from commercial contracts, and both research and vendor communities have advocated multi-agent architectures as a superior alternative to single-prompt deployments. The empirical case for this advocacy is, however, thin: existing legal multi-agent systems each evaluate a single architecture on a single benchmark, no published work has compared multi-agent patterns to single-agent baselines across multiple frontier-model families, and recent cross-domain work suggests that multi-agent gains diminish as base-model capability rises.

This thesis presents the first controlled cross-architecture cross-model comparison of multi-agent designs for clause extraction on the CUAD benchmark. Five configurations spanning zero-shot, chain-of-thought, pipeline decomposition, LLM-routed specialists, and a consolidated single-agent ablation are evaluated across eleven frontier models from three providers, yielding 163,644 paired predictions analysed with paired statistical tests at the natural class distribution.

No multi-agent configuration consistently outperforms the strongest single-agent baseline. Pooled accuracy across the eleven models falls within a one-point band, and architectural ranking varies with model scale rather than model family: larger models perform best under simpler baselines, while lighter models gain from structured decomposition. Two mechanisms explain the null result. The pipeline architecture exhibits a *classification-gate rejection cascade* — its upstream classifier silently overrejects on some models — and the LLM router in the specialist architecture is empirically redundant: it converges to a deterministic category lookup and produces lower-quality span outputs than the consolidated ablation. Multi-agent configurations carry a 24%–70% cost premium without aggregate accuracy benefit.

These results delineate when multi-agent designs help, are inert, or actively hurt in enterprise CLM, and motivate a practitioner-oriented analysis of the gap between benchmark performance and production deployment.

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Research Gap . . . 1
1.3 Research Questions . . . 2
1.4 Hypotheses . . . 3
1.5 Contributions . . . 3
1.6 Thesis Outline . . . 4

2 Background and Related Work **5**
2.1 Background . . . 5
2.1.1 Contract Lifecycle Management . . . 5
2.1.2 Large Language Models . . . 6
2.1.3 Agentic AI and Multi-Agent Systems . . . 7
2.1.4 The CUAD Benchmark . . . 8
2.2 Contract Analysis Methods . . . 9
2.3 Multi-Agent Legal AI . . . 11
2.4 Conditions for Multi-Agent Effectiveness . . . 12
2.5 Evaluation Methodology . . . 13
2.6 Research Gap . . . 15

3 Methodology **17**
3.1 Research Design . . . 17
3.2 Dataset and Sampling . . . 18
3.3 Model Selection . . . 19
3.4 Metrics . . . 20
3.5 Hypotheses . . . 21
3.6 Statistical Methodology . . . 21
3.7 Threats to Validity . . . 23

4 System Design and Implementation **25**
4.1 System Components and Data Flow 25
4.2 Implementation Stack 25
4.3 Shared Abstractions 26
4.4 Baseline Architectures 27
4.5 Multi-Agent Architectures 28
4.6 Observability and Trace Schema 31
4.7 Experimental Infrastructure 32

5 Results **35**
5.1 Run Coverage and Sample Accounting 35
5.2 H1 — Multi-Agent vs. Single-Agent Baselines 36
5.3 H2 — Rare-Category Gap 37
5.4 H3 — Architecture vs. Prompts (M1 vs. M6) 38
5.5 Additional Findings 39
5.5.1 The Classification-Gate Rejection Cascade 39
5.5.2 Cost-Efficiency Analysis 40

6 Discussion **43**
6.1 When Multi-Agent Helps: Evidence for Pipeline Decomposition 43
6.2 When Multi-Agent Hurts: The LLM-Router Problem 43
6.3 Architecture vs Prompts: The H3 Verdict 43
6.4 Practitioner Perspective: Gaps Between Benchmarks and Real-World CLM 43
6.5 Connecting to Prior Work 43
6.6 Limitations 43

7 Conclusion and Outlook **44**
7.1 Summary of Contributions 44
7.2 Answers to Research Questions 44
7.3 Outlook and Future Work 44

List of Figures **45**

List of Tables **47**

Bibliography **49**

1 Introduction

1.1 Motivation

Contract Lifecycle Management (CLM) is the workflow through which an organisation authors, negotiates, executes, monitors, and retires its contractual instruments. The economic surface is large: organisations forfeit value equivalent to roughly 9% of annual revenue through poor contracting practices, with contract data dispersed across an average of 24 systems and 83% of executives reporting their contracts too rigid to accommodate change [Wor25]. Goldman Sachs estimate 44% of U.S. legal-task time is technically automatable by generative AI — second-highest exposure across all professions [Hat+23].

Frontier large language models (LLMs) match junior lawyers on issue-identification F-scores (0.87 vs. 0.88) while reducing review time by $\approx 100\times$ [Mar+24], and independent benchmarks confirm competitiveness on standardised clause-extraction tasks [Hen+21; Liu+25]. Single-prompt deployments nonetheless dominate commercial CLM, and three production failures persist: hallucination, prompt sensitivity [Cha+24], and laziness — the silent default to "no related clause" even when the clause is present in the contract [Liu+25].

Multi-agent architectures are the proposed remedy. Decomposing clause analysis into specialised agents promises improved recall through specialisation, improved explainability through inspectable inter-agent traces, and addresses transparency and traceability requirements under Article 13 of the EU AI Act [EC24] and Article 22 of the GDPR [EC16]. Frameworks including AutoGen [Wu+23], MetaGPT [Hon+24], and Mixture-of-Agents [Wan+24] have made such architectures cheap to assemble. Whether they actually improve contract analysis is, however, an open empirical question.

1.2 Research Gap

Three observations motivate the gap this thesis addresses.

First, existing legal multi-agent systems — ChatLaw [Cui+24], PAKTON [Rap+25], L-MARS [WY25] — each evaluate a single architecture on a single benchmark without ablating the architectural choice itself. The empirical question of which multi-agent pattern is preferable for clause-level contract analysis under a fixed model and task remains empirically unresolved.

Second, recent cross-domain work questions whether multi-agent designs help at all once base-model capability is high. Gao et al. show diminishing returns for frontier models whose long-context reasoning absorbs the workload that previously required decomposition [Gao+25]. Cemri et al. construct a 14-mode failure taxonomy across seven multi-agent frameworks and report only modest 14–15% recovery from simple interventions, suggesting structural rather than prompt-level causes [Cem+25]. Bogavelli et al. find no universally optimal architecture across frontier models on enterprise tasks, with model–architecture interaction dominating absolute architectural ranking [Bog+25].

Third, prior CUAD evaluations have used inconsistent sampling designs. Evaluations at inflated positive ratios produce optimistic precision estimates because few absent samples are tested, masking the dual failure mode of over-extraction.

No published study simultaneously (i) cross-varies multi-agent architecture and frontier-model family on CUAD, (ii) samples at the natural class distribution, (iii) applies paired statistical tests with multiple-comparison correction, and (iv) ablates the multi-agent design against a consolidated single-agent prompt. This thesis addresses all four requirements.

1.3 Research Questions

Main RQ. Under what conditions do multi-agent architectures improve contract analysis accuracy, efficiency, and explainability in enterprise CLM systems, and which architectural patterns are most effective?

RQ1. Under what model-family and prevalence conditions does each architectural pattern — pipeline decomposition, specialist routing, or consolidated prompting — achieve the highest F_2?

RQ2. How do multi-agent configurations compare to single-agent baselines in F_2 and provider cost across clause categories and contract lengths?

RQ3. Which architectural decisions produce auditable reasoning traces, and what tradeoffs do they introduce?

1.4 Hypotheses

H1. At least one multi-agent configuration outperforms the strongest single-agent baseline on F_2 (McNemar exact, α=0.05, BH-corrected).

H2. Multi-agent configurations reduce the rare–common F_2 gap relative to single-agent baselines (paired bootstrap on gap difference, 95% CI).

H3. The LLM-routed specialist system M1 is statistically distinguishable from the consolidated specialist ablation M6 on F_2 (two-sided McNemar exact).

RQ3 is addressed by system design rather than statistical hypothesis test; the auditability properties of the implementation are documented in §4.6.

1.5 Contributions

C1 **Cross-architecture benchmark.** The first cross-family cross-architecture comparison on CUAD: 5 architectural configurations $\times$ 11 frontier models, yielding 163,644 paired sample-level predictions analysed with McNemar exact tests, paired bootstrap, and Benjamini–Hochberg correction.

C2 **Model-scale gradient.** Architectural ranking is not constant across the model panel: larger models achieve their best F_2 under simpler single-agent baselines, while lighter or weaker models gain from structured decomposition. No architecture is universally optimal — pooled F_2 across all five configurations spans a 0.011-point band.

C3 **Classification-gate cascade.** A failure mode of pipeline-decomposition multi-agent architectures, named and quantified: the upstream classifier accounts for the majority of false negatives, propagating silent rejections downstream. The cascade is severe — Gemini 2.5 Flash's wrong-rejection rate jumps from 2.7% under a single-agent baseline to 40.9% in the pipeline — and its direction depends on the base model's laziness profile, harming low-laziness models and helping high-laziness ones, making the effect predictable from a single-agent baseline run.

C4 **Routing redundancy with span-quality degradation.** The LLM router in the specialist architecture is empirically redundant: it agrees with a deterministic category-to-specialist mapping on every model in the panel — 100% routing accuracy on 10 of 11 models — and the consolidated single-agent ablation matches it on F_2 while producing cleaner spans. The routing topology adds cost without architectural benefit.

A methodological observation underpins all four contributions. The thesis samples at CUAD's natural 30%/70% positive/negative class ratio rather than the inflated positive ratios used in some prior work, exposing over-extraction as the dominant zero-shot failure mode that positive-heavy sampling masks (§3.2).

1.6 Thesis Outline

Chapter 2 surveys background and related work. Chapter 3 specifies methodology. Chapter 4 documents the implementation. Chapter 5 reports empirical results. Chapters 6 and 7 discuss findings and conclude.

2 Background and Related Work

2.1 Background

2.1.1 Contract Lifecycle Management

Contract Lifecycle Management (CLM) denotes the systematic management of a contractual agreement across six canonical stages: request, author, negotiate, execute, monitor, and renew or retire [Wor25]. Each transition produces auditable artefacts — drafts, redlines, signature blocks, obligation registers — and each stage admits distinct algorithmic interventions, from clause assembly at the authoring stage to obligation extraction at the monitoring stage. Figure 2.1 visualises the lifecycle; the NLP-heavy stages are highlighted.

World Commerce & Contracting estimates that organisations forfeit $\approx 9\%$ of annual contract value through poor contract management, with detection-stage failures (missed clauses, unobserved obligations, undetected deviations) dominating the loss distribution [Wor25]. Goldman Sachs places the total addressable population of U.S. legal-services tasks exposed to generative AI at approximately 44%, with contract review repeatedly identified as the highest-leverage subset [Hat+23].

The commercial CLM market is consolidated around platforms whose positioning is documented in successive Gartner Magic Quadrants. Sirion, Icertis, Ironclad, and DocuSign are the four vendors consistently designated Leaders, each having rebuilt portions of their analysis stack on LLMs since 2023. Sirion markets an Extraction Agent and a Redline Agent under an "agentic" architecture. Icertis ships ICI Copilots backed by the Microsoft Azure OpenAI Service. Ironclad embeds AI Assist for redlining and Repository AI for post-signature search. DocuSign closed its US$165M acquisition of Lexion in May 2024, folding Lexion's analytics into the Intelligent Agreement Management (IAM) platform. Adjacent vendors include Spellbook (in-Word drafting on GPT-4 class models) and Harvey, whose contract-analysis workflow serves A&O Shearman and PwC [AO 23; PwC23]. Across this set, public product documentation describes single-prompt LLM calls, retrieval-augmented prompts, or task-specific

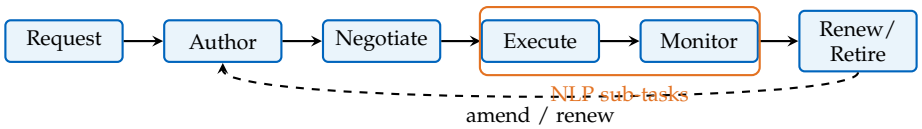


Figure 2.1: CLM lifecycle. Clause extraction, risk identification, and obligation tracking (highlighted) occur primarily in the Execute and Monitor stages.

fine-tunes; no Leader has publicly disclosed a deployed multi-agent architecture for clause-level analysis.

2.1.2 Large Language Models

An LLM is a neural language model parameterised by θ that factorises a token sequence $\mathbf{x} = (x_1, \ldots, x_n)$ autoregressively:

$$p_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}). \tag{2.1}$$

Training minimises the negative log-likelihood $\mathcal{L}(\theta) = -\sum_t \log p_\theta(x_t \mid x_{<t})$ over large text corpora following the Transformer derivation [Dev+19]. Modern frontier LLMs extend Eq. (2.1) with three post-training layers decisive for agentic deployment: supervised instruction tuning, preference alignment via reinforcement learning from human or AI feedback, and structured-output fine-tuning that turns free-form generation into validated JSON and tool calls.

Table 3.2 summarises the frontier-model releases relevant to the experimental envelope of this thesis.

A parallel layer of legal-domain products consumes these frontier models. Harvey serves A&O Shearman, PwC, O'Melveny & Myers, and Macfarlanes, and was reported in 2025 at valuations rising from US$3B to US$11B as it expanded into Vault-based document review and Agent Builder workflows [AO 23; PwC23]. Lexis+ AI and Thomson Reuters CoCounsel embed retrieval-augmented generation over proprietary legal corpora. None of these systems publishes peer-reviewed evaluations of clause-level extraction on public benchmarks. ContractEval [Liu+25] is, to our knowledge, the only large-scale academic evaluation of LLMs on contract clause extraction conducted on a public benchmark with reproducible methodology.

Model	Provider	Release	Context	Notes
GPT-4	OpenAI	Mar 2023	8K / 32K	Multimodal preview, RLHF tuned
GPT-4 Turbo	OpenAI	Nov 2023	128K	Function calling GA, JSON mode
GPT-4o	OpenAI	May 2024	128K	Native multimodal I/O
GPT-5	OpenAI	Aug 2025	400K	Unified router over fast/-thinking modes
Claude 3 Opus	Anthropic	Mar 2024	200K	Tool use API
Claude 3.5 Sonnet	Anthropic	Jun 2024	200K	Computer-use beta, artifacts
Claude Sonnet 4	Anthropic	May 2025	200K	Extended thinking; MCP code-exec tools
Claude Sonnet 4.5	Anthropic	Sep 2025	200K (1M beta)	Agentic/tool-orchestration focus
Claude Haiku 4.5	Anthropic	Oct 2025	200K	Cost-optimised agentic tier
Claude Opus 4.5	Anthropic	Nov 2025	200K	SWE-bench Verified >80%
Gemini 1.5 Pro	Google	Feb 2024	1M	Mixture-of-experts, long-context
Gemini 2.0 Flash	Google	Dec 2024	1M	Multimodal Live API
Gemini 2.5 Pro	Google	Jun 2025	1M	Thinking budget control, MCP support
Gemini 3 Pro	Google	Nov 2025	1M	Agentic/tool-use frontier

Table 2.1: Frontier LLM releases referenced in this thesis. Context-window figures denote the standard production context; beta long-context modes noted parenthetically. Capabilities summarised from official announcements.

2.1.3 Agentic AI and Multi-Agent Systems

An LLM-based agent is a system in which one or more LLM calls are wrapped in a control loop that observes state, plans, invokes tools, and updates memory [Yao+23b; Wei+22]. A Multi-Agent System (MAS) replaces a single such loop with multiple cooperating loops, each parameterised by a role, a tool surface, and (often) a different model. Five design axes locate any MAS in the design space; Figure 2.2 maps the options and marks those ablated in this thesis.

The empirical literature has converged on five design axes along which any MAS can be located. *Topology* distinguishes hierarchical configurations (orchestrator dispatches to subordinate workers) from peer configurations (agents communicate without a central coordinator), as in MetaGPT's role-pipeline [Hon+24] and CAMEL's role-playing

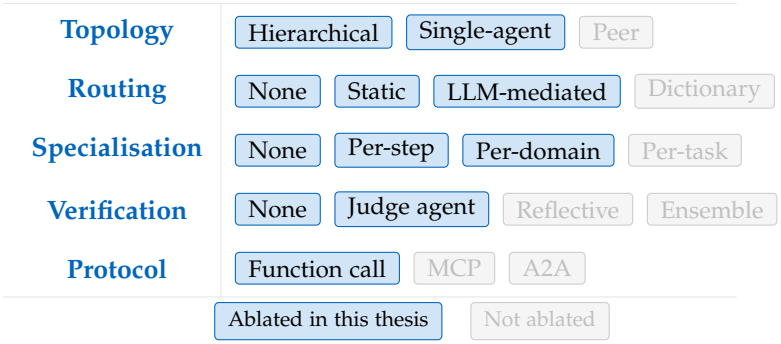


Figure 2.2: Multi-agent design space: five structural axes and their principal options. Shaded boxes mark options covered by at least one experimental configuration (B1, B4, M0, M1, M6) in this thesis. Chapter 3 specifies which configuration selects which option.

pairs [Li+23]. *Routing* distinguishes static pipelines (fixed step order), LLM-mediated routing (a router agent selects a downstream worker), and dictionary-based routing (a deterministic feature-to-worker map). *Specialisation granularity* ranges from per-task agents (one per CUAD category) through per-domain agents (one for risk, IP, temporal) to per-step agents (extract, verify, format). *Verification structure* ranges from none, through reflective self-critique [Shi+23], multi-agent debate, judge-agent verification, tree search [Yao+23a], majority-vote ensembles such as Mixture-of-Agents [Wan+24], and self-consistency [Wan+23b]. The *communication protocol* is the lowest layer: function calling via vendor APIs, the Model Context Protocol (MCP) released by Anthropic in November 2024 [Ant24], the Agent2Agent (A2A) protocol released by Google in April 2025 and donated to the Linux Foundation in June 2025 [Sur+25], or framework-specific structured messages.

Production-relevant orchestrators include AutoGen [Wu+23], MetaGPT [Hon+24], LangGraph [Lan24a], CrewAI, and OpenAI's Agents SDK. The five axes — topology, routing, specialisation granularity, verification structure, and communication protocol — define the design space whose CLM-relevant slice this thesis ablates in Chapter 3.

2.1.4 The CUAD Benchmark

The Contract Understanding Atticus Dataset (CUAD) is the canonical public benchmark for clause-level contract review [Hen+21]. It consists of 510 commercial contracts annotated against 41 categories of legal clauses. The annotation pipeline employed law

students who completed 70–100 hours of contract-review training under the supervision of practising attorneys [Hen+21], a provenance restated by ContractEval as a strength relative to crowd-sourced legal datasets [Liu+25]. Table 2.2 reproduces the dataset statistics.

Property	**Value**
Contracts	510
Clause categories	41
Expert annotations	>13,000 spans
Source	EDGAR commercial filings
Train / test split	399 / 102
Task formulation	Span extraction + binary presence
License	CC BY 4.0
Long contracts (>100k chars)	81 (used in this thesis)

Table 2.2: CUAD dataset statistics [Hen+21].

The instance count is highly imbalanced: at the (contract, category) level, the natural prevalence of positive instances is ≈30%, a structural property of contract review rather than a sampling artefact. This thesis evaluates on the long-contract subset comprising 81 contracts whose raw character length exceeds 100,000 characters. Two considerations motivate this choice: first, the long-contract regime is where context-window pressure on a single LLM call is highest, so any putative benefit of architectural decomposition should be most visible; second, concentrating on long contracts produces an architecturally more demanding evaluation envelope than the full CUAD distribution. Figure 2.3 illustrates the F_1 gradient across tiers that motivates the domain-specialisation hypothesis (H2).

2.2 Contract Analysis Methods

The methodological lineage of automated contract analysis runs from rule-based extractors through statistical sequence labellers to fine-tuned and prompted Transformers. Early systems used hand-written regular expressions and gazetteers over POS-tagged text. Linear-chain Conditional Random Fields [LMP01] and BiLSTM–CRF taggers [Lam+16] formalised the task as token-level sequence labelling and dominated the LEDGAR [Tug+20] and ContractNLI [KM21] eras. The release of BERT [Dev+19] and legal-domain LEGAL-BERT [Cha+20] shifted the regime from feature engineering to pre-train-and-fine-tune. CUAD's reference numbers were obtained with RoBERTa-style

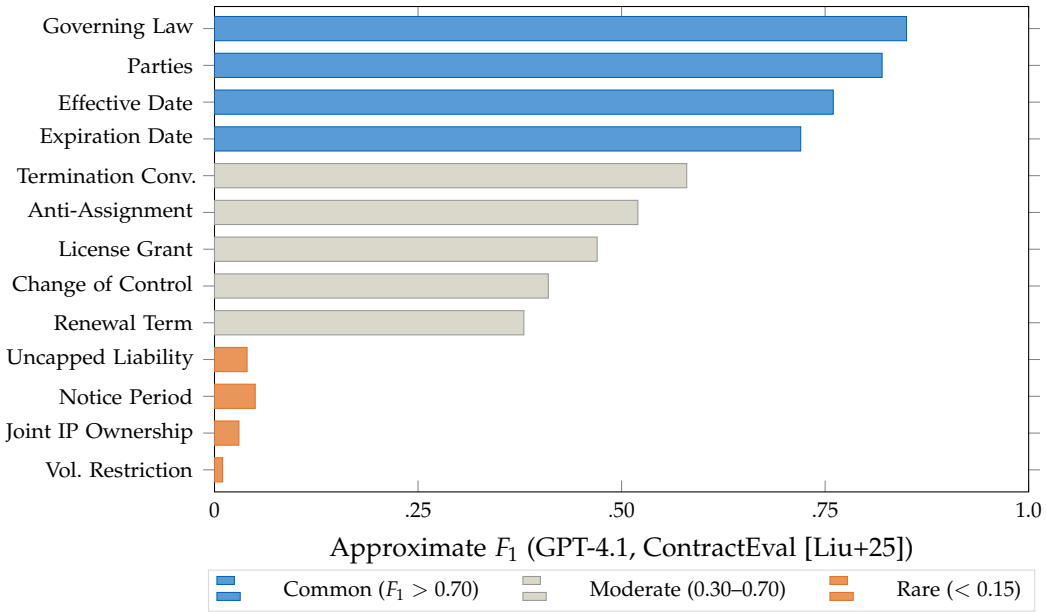


Figure 2.3: Approximate F_1 scores for 13 representative CUAD categories under GPT-4.1 (ContractEval [Liu+25]). Values are indicative; exact scores vary by model. Rare-tier categories achieve near-zero F_1 under the strongest single-agent baseline, directly motivating the per-domain specialist design evaluated in Chapter 3.

extractive question answering [Hen+21]. MAUD [Wan+23a] extended the paradigm to merger-agreement understanding.

The LLM-prompting regime is characterised by two reproducible studies. Better Call GPT [Mar+24] reported that GPT-4 with structured prompting can match or exceed human associates on contract-review subtasks under controlled conditions. ContractEval [Liu+25] evaluated 19 LLMs (4 proprietary, 15 open-source) on CUAD, found a persistent gap between proprietary and open-source models, and identified the named failure mode of *laziness* — the model declines to extract any span when the clause is present. ContractEval also documented a tension between reasoning mode and correctness: chain-of-thought-style decoding improved Jaccard span overlap on positive instances but degraded overall correctness, particularly on negative instances.

Table 2.3 compares the main contract-analysis benchmarks.

Dataset	Task	Size	Categories	Used here
CUAD [Hen+21]	Span extraction	510 contracts	41	✓
ContractNLI [KM21]	Doc-level NLI	607 NDAs	17	—
MAUD [Wan+23a]	Reading comp.	152 M&A agmt.	152	—
LEDGAR [Tug+20]	Multi-label cls.	60k contracts	100+	—

Table 2.3: Main contract-analysis benchmarks.

Why multi-agent for contract analysis. Four properties of the CUAD-style problem motivate ablating multi-agent architectures on this task. First, contracts in the long-context subset routinely exceed 100,000 characters; explicit decomposition makes per-call context tractable where single-prompt formulations must truncate or chunk silently. Second, the 41 CUAD categories partition into risk-oriented, IP/commercial, temporal, and consideration-oriented clusters; a specialisation hypothesis predicts that role-conditioned agents may outperform a single generalist on within-cluster categories. Third, legal review is recall-asymmetric: a missed clause exposes the client to undisclosed obligations whereas a spurious extraction is corrected during attorney review at lower cost — motivating F_2-weighted evaluation (Section 2.5) and architectures that boost recall at controlled precision cost. Fourth, regulated environments — the EU AI Act treats certain legal AI uses as high-risk [EC24] and GDPR requires data-protection accountability [EC16] — demand traceable per-decision provenance; multi-agent architectures with explicit verifier or judge agents produce that trace at the architectural level.

2.3 Multi-Agent Legal AI

Table 2.4 codes the published multi-agent legal systems most directly comparable to this thesis along the five design axes introduced in Section 2.1.3.

ChatLaw v2 [Cui+24] couples a Mixture-of-Experts router with role-conditioned domain agents for Chinese legal consultation; verification is a self-check pass and there is no public ablation against single-agent variants. PAKTON [Rap+25] pipelines a reading agent, an analysis agent, and a drafting agent over open-ended contract-review prompts; the system is evaluated qualitatively and not on CUAD. L-MARS [WY25] combines a planner, a retriever, and a judge agent on statutory research; performance is reported at system level without architectural decomposition. Lawma [Dom+24], while not multi-agent, provides the strongest counter-evidence to architectural complexity in the

System	Task	Topology	Routing	Specialisation	Verification
ChatLaw v2 [Cui+24]	Chinese legal QA	Hierarchical	LLM-mediated	Per-domain	Self-check
PAKTON [Rap+25]	Contract review	Hierarchical	Static	Per-step (3)	None
L-MARS [WY25]	Statutory research	Hierarchical	LLM-mediated	Per-step + retrieval	Judge agent
Lawma [Dom+24]	Legal annotation	Single-agent	N/A	Per-task (fine-tune)	None
Layered CoT [San25]	Statutory reasoning	Single-agent	Static	Per-step CoT layers	Self-consistency
This thesis	CUAD long-contract	Both	All three	All three	Reflexion + judge

Table 2.4: Multi-agent legal AI systems coded against the five design-space axes of Section 2.1.3. All prior systems evaluate a single architecture without ablation.

legal domain: a fine-tuned 8B Llama 3 model exceeds GPT-4 by 20.0 accuracy points on legal annotation, and the smallest variant (135M) is competitive with Claude 3.7 Sonnet. The layered chain-of-thought system of Sanwal [San25] stays within a single agent but stacks specialist prompts and self-consistency, reporting gains over flat prompting on statutory reasoning.

Across this set, every prior system reports performance for a single chosen architecture without ablating it against simpler or alternative configurations on the same evaluation envelope. The question of whether the architecture or the model is responsible for observed performance therefore remains open. This thesis addresses it directly through cross-architecture ablation on a fixed CUAD evaluation harness.

2.4 Conditions for Multi-Agent Effectiveness

Four recent studies bound the conditions under which multi-agent systems produce gains over single-agent baselines. Gao et al. [Gao+25] show on a battery of reasoning and tool-use tasks that single, well-prompted LLMs frequently match or exceed multi-agent configurations once the latter are corrected for cost and prompting parity, and that observed multi-agent gains often disappear under stronger single-agent baselines. Cemri et al. [Cem+25] taxonomise multi-agent failure modes — specification drift between agents, inter-agent verification gaps, lossy summarisation across hand-offs — and document that these failures account for the majority of regressions in the systems

they audit. Bogavelli et al. [Bog+25] run a large architectural sweep across orchestration patterns and base models and find substantial model-family × architecture interactions: an architecture that is optimal under one model family is often dominated under another. Sanwal [San25] demonstrates that layered single-agent chain-of-thought can recover much of the benefit attributed to multi-agent decomposition without paying the coordination cost.

Synthesised against the working hypotheses of this thesis: (H1) a generic claim of multi-agent superiority on CUAD long-contract extraction is unlikely to survive a properly normalised single-agent baseline [Gao+25], so the hypothesis is stated and evaluated as conditional; (H2) the relative ordering of architectures will depend on the underlying model family [Bog+25], encoded as an explicit interaction term in the experimental design; and (H3) classification-gate cascades — a router agent that decides whether to extract before invoking an extractor — are a natural counter to the verification-gap and laziness failures catalogued in [Cem+25], motivating the M0 ablation in Chapter 3.

2.5 Evaluation Methodology

This section consolidates the evaluation primitives used throughout the thesis. Chapter 3 references these definitions without restating them.

Span and instance metrics. For a category c on a contract d, let $S^*_{c,d}$ be the set of gold spans and $\hat{S}_{c,d}$ the set of predicted spans. A predicted span $\hat{s}$ matches a gold span s when their character-level Jaccard overlap exceeds threshold τ=0.5 following [Hen+21]:

$$J(\hat{s}, s) = \frac{|\hat{s} \cap s|}{|\hat{s} \cup s|}. \tag{2.2}$$

At the (contract, category) level, the prediction is true-positive if any predicted span matches any gold span at threshold τ, false-negative if gold spans exist with no match, false-positive if predicted spans exist with no match, and true-negative if both sets are empty. Precision P and recall R aggregate over the natural class distribution.

F_β. Recall is privileged over precision in legal review by a factor commonly fixed at β=2:

$$F_\beta = \frac{(1+\beta^2)\, P\, R}{\beta^2 P + R}, \qquad \beta = 2. \tag{2.3}$$

F_1 is reported alongside F_2 for compatibility with prior CUAD work. Instance-level F_β at the (contract, category) granularity is the primary metric reported in Chapter 4, as it aligns with the deployment unit (a reviewer's decision per category per contract).

Failure-mode metrics. ContractEval [Liu+25] introduced *laziness* as the per-model rate at which an LLM declines to extract on instances where a gold span exists:

$$\lambda = \frac{|\{(c,d) : |\hat{S}_{c,d}| = 0 \ \wedge \ |S^*_{c,d}| > 0\}|}{|\{(c,d) : |S^*_{c,d}| > 0\}|}. \tag{2.4}$$

The dual failure mode is *over-extraction* — the per-model rate of producing spans on negative instances:

$$\omega = \frac{|\{(c,d) : |\hat{S}_{c,d}| > 0 \ \wedge \ |S^*_{c,d}| = 0\}|}{|\{(c,d) : |S^*_{c,d}| = 0\}|}. \tag{2.5}$$

Equations (2.4) and (2.5) are a coupled pair: evaluations that resample to a balanced class distribution suppress Eq. (2.5) and hide the precision cost of recall-boosting interventions. We therefore evaluate exclusively on the natural CUAD class distribution and report both metrics.

Statistical testing. We pair every architecture comparison on the same contract set and report McNemar's test for instance-level binary outcomes [McN47] and the paired bootstrap with B=10,000 resamples for span-level continuous metrics [Efr79; RM05]. Multiple-comparison control across the architecture grid uses Benjamini–Hochberg false-discovery-rate correction at q=0.05 [BH95]. Sample-size and power considerations follow Dror et al. [Dro+18] and Card et al. [Car+20]; the 81-contract subset gives ≥ 0.8 power to detect absolute F_2 differences of $\approx$2.5 percentage points at the instance level under the bootstrap. Implementations use SciPy [Vir+20] and NumPy [Har+20].

Sampling design. CUAD's natural clause-presence rate is $\approx$ 30% positive at the instance level. Resampling to balance positive and negative instances inflates apparent precision and suppresses Eq. (2.5), detaching the evaluation from any deployable distribution where 41 categories are scanned per contract and most are absent. We retain natural prevalence throughout [Liu+25].

ContractEval as direct empirical predecessor. ContractEval [Liu+25] evaluates 19 LLMs (4 proprietary, 15 open-source) on 4,128 (contract, category) samples across the full 41-category CUAD schema, holding prompt and decoding constants across models, and reporting Jaccard span overlap, instance-level correctness, and laziness. Its principal findings are: a persistent proprietary–open-source gap on legal extraction; a non-monotonic effect of explicit reasoning (chain-of-thought improves Jaccard on positive instances while degrading correctness through inflated false positives); and laziness as a robust, model-dependent and category-dependent failure mode. Three properties of the ContractEval design define the gaps the present thesis closes: the architectural axis is held fixed at single-prompt single-model; model-family interactions are not analysed; and over-extraction (Eq. (2.5)) is not tracked as a first-class metric. We retain ContractEval's evaluation protocol — natural class distribution, instance-level scoring, laziness reporting, τ=0.5 Jaccard threshold — and extend it along the architectural axis and the dual failure-mode axis.

Prompt sensitivity as a validity threat. POSIX [Cha+24] quantifies the sensitivity of LLM outputs to semantically equivalent prompt rewrites, reporting non-trivial sensitivity even on well-defined tasks. Prompt sensitivity is therefore a first-order validity threat for any architecture comparison in which architectures differ in the number, role, or wording of agent prompts. Chapter 3 controls this threat by holding system-prompt scaffolds fixed across architectures where the role permits, and reports a POSIX-style sensitivity check on the architectural ablations. Observability and trace inspection use Langfuse [Lan24b]; orchestration uses LangGraph [Lan24a].

2.6 Research Gap

Four gaps in the published record jointly motivate this thesis. (i) No published study performs a controlled cross-architecture ablation on CUAD-class clause extraction; the closest predecessor [Liu+25] varies model identity only. (ii) No published study reports a model-family $\times$ architecture interaction analysis on this task, despite the prediction in [Bog+25] that such interactions dominate single-axis effects. (iii) No published study tracks both Eq. (2.4) and Eq. (2.5) under the natural class distribution, leaving the recall–precision asymmetry intrinsic to legal review unobserved. (iv) Production CLM platforms — Sirion, Icertis, Ironclad, DocuSign — disclose single-prompt or retrieval-augmented LLM deployments with no public multi-agent ablation, leaving the architectural design space untested in the regime where the question is commercially decisive.

We design and evaluate a slice of the multi-agent design space (topology $\times$ routing $\times$ specialisation $\times$ verification) on the long-contract CUAD subset under the ContractEval protocol extended with Eq. (2.5) and a model-family interaction term, in order to determine — with statistical control across the architecture grid [BH95; Dro+18] — under which conditions multi-agent decomposition is justified for CLM deployment and under which conditions it is dominated by a stronger single-agent baseline.

3 Methodology

This chapter specifies the experimental design used to evaluate multi-agent architectures for clause extraction in CLM. The design is a controlled comparison across the architecture grid introduced in Chapter 2, executed against a panel of frontier LLMs on the long-contract subset of CUAD under the natural class distribution. RQ3 (auditable reasoning traces) is addressed by system design rather than statistical hypothesis test; the auditability properties of the implementation are documented in Section 4.6.

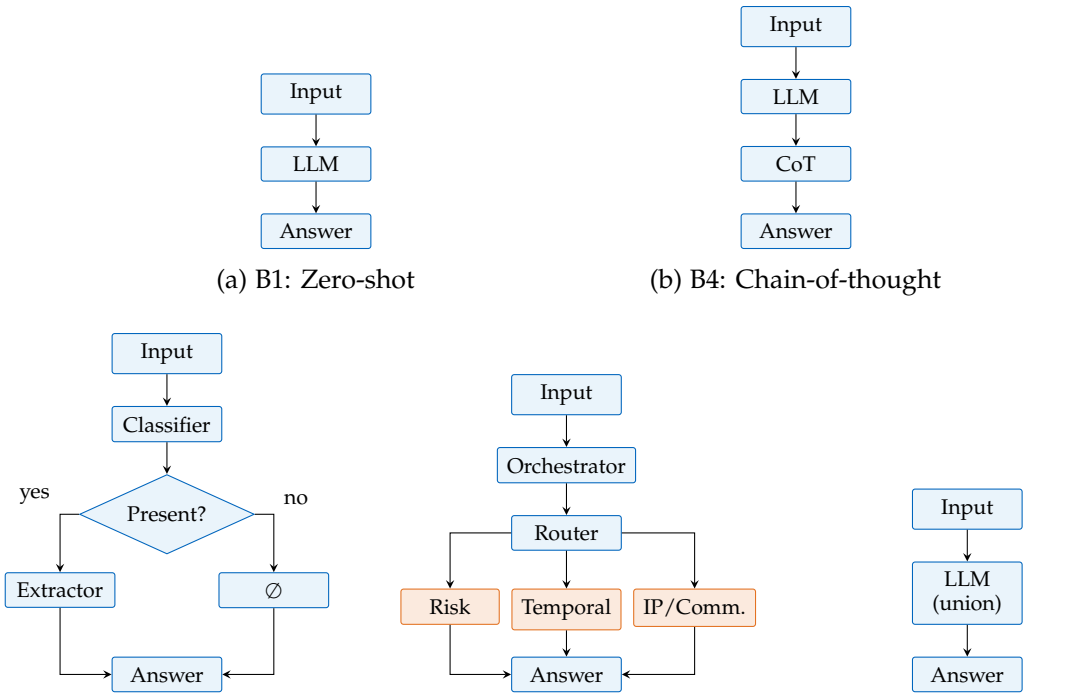
3.1 Research Design

Five configurations are evaluated against 11 frontier LLMs on 81 long CUAD contracts (>100,000 chars), yielding $5 \times 11 = 55$ official runs and 163,644 analysed predictions. Differences between configurations are minimised to a single architectural axis per comparison (Table 3.1), following the controlled comparative protocol of [Dro+18; Car+20]. Every statistical test is paired on the intersection of sample IDs, so performance differences are attributable to architecture or model, not dataset drift.

ID	Name	Stages	Routing	Specialisation
B1	Zero-shot single agent	1	None	Generalist
B4	Chain-of-thought single agent	1	None	Generalist + CoT
M0	Classify-then-extract pipeline	2	Static gate	Generalist
M1	Orchestrator + 3 specialists	2	LLM router	Three domain experts
M6	Consolidated specialist (ablation)	1	None	Union of M1 prompts

Table 3.1: Five evaluated configurations. M0 vs. B1 isolates the classification gate; M1 vs. M6 isolates routing and decomposition; B4 vs. B1 isolates chain-of-thought.

Figure 3.1 shows the message-flow diagram for each configuration.



(a) B1: Zero-shot (b) B4: Chain-of-thought

(c) M0: Classify-then-extract (d) M1: Router + specialists (e) M6: Ablation

Figure 3.1: Message-flow diagrams for the five configurations. Input = contract text + clause-category indicator. Blue nodes: generalist LLM invocations. Orange nodes: domain-specialist prompts. ∅ = silent no-clause return.

3.2 Dataset and Sampling

CUAD [Hen+21] is the evaluation corpus (see Table 2.2 for statistics). We restrict to 81 contracts >100,000 characters because long-context performance is the regime where decomposition is hypothesised to add value and approximates the operational distribution of CLM platforms surveyed in Section **??**. The 81-contract subset yields $81 \times 41 = 3{,}321$ raw (contract, category) pairs per evaluation pass.

Sampling strategy. A `neg_ratio=0.7` sampler produces a 30%/70% positive/negative split per run, mirroring CUAD's natural class prior at the (contract, category) level. Two design considerations motivate this choice. First, ContractEval evaluates 4,128 instances drawn at the natural distribution, and matching that protocol enables direct comparability with the closest empirical predecessor [Liu+25]. Second, prior work

that resamples to inflated positive ratios suppresses the false-positive denominator and masks over-extraction (Eq. 2.5), detaching the evaluation distribution from any deployable distribution. The natural prior preserves both laziness (Eq. 2.4) and over-extraction (Eq. 2.5) as observable quantities. Sampling is seeded (`seed=42`) so the same (contract, category) pairs are evaluated across all configurations, making every statistical test paired at the instance level. Each run yields ≈2,999 predictions; 1,297 of 164,941 total (0.79%) were excluded for malformed classification tokens.

Prevalence tiers. The 41 CUAD categories are partitioned into three tiers — *common*, *moderate*, *rare* — by natural positive rate in the 81-contract subset. Tier cutpoints are fixed before any model is run.

3.3 Model Selection

Table 3.2 lists the 11 frontier proprietary models in the panel. Selection follows three criteria. First, three families span the providers used in commercial CLM (Anthropic, Google, OpenAI), so model-family effects predicted by [Bog+25] can be observed. Second, within each family the panel spans a compute-tier range from light (Haiku, Flash, Nano) to heavy (Opus, Pro), so capability × architecture interactions can be observed. Third, model identifiers are pinned to specific snapshots in the run metadata for reproducibility.

Family	Model	Tier
Anthropic	Claude Haiku 4.5	Light
	Claude Sonnet 4	Mid
	Claude Sonnet 4.6	Mid
	Claude Opus 4.6	Heavy
Google	Gemini 2.5 Flash	Light
	Gemini 2.5 Pro	Heavy
	Gemini 3 Flash	Light
	Gemini 3.1 Pro	Heavy
OpenAI	GPT-4.1	Mid
	GPT-4.1 Mini	Light
	GPT-4.1 Nano	Light

Table 3.2: Eleven frontier proprietary models evaluated. Three families; compute-tier range covers light-to-heavy within each family.

Open-weights models are excluded from the main panel for three reasons. First, the commercial CLM platforms surveyed in Section **??** deploy proprietary APIs, and the thesis's external-validity claim is restricted to that deployment population. Second, the long-contract subset requires context windows of $\geq$ 128K tokens, which most open-weights models do not support. Third, ContractEval's 15-model open-weights evaluation [Liu+25] documents a persistent proprietary-vs-open-source gap on extraction correctness; pilot work for this thesis (not reported) confirmed that the gap does not invert under multi-agent decomposition. Open-weights extension is identified as future work in Chapter 7.

3.4 Metrics

All metrics are formally defined in Section 2.5 (Equations 2.3–2.5).

F_2 **(primary).** β=2 weights recall 4$\times$ over precision. The asymmetry is justified by the deployment cost structure of CLM: a missed clause passes undetected into a signed contract, whereas a spurious extraction is caught by a reviewing lawyer. F_2 is the primary hypothesis-testing metric and the basis for H1–H3.

F_1 **(secondary).** Equal-weight F-score, reported for comparability with prior work — ContractEval reports F_1 as its headline metric [Liu+25].

Jaccard similarity (span quality). $|A \cap B|/|A \cup B|$ over token sets, computed conditional on a non-empty prediction. Because the F-score TP criterion requires full span coverage, partial extractions that cover most of a clause register as false negatives; Jaccard gives partial credit and diagnoses near-miss behaviour independently of the binary correctness signal.

Laziness rate λ (diagnostic). Fraction of positive-label instances on which the model returns "no related clause". The dominant failure mode of single-agent LLMs on rare CUAD categories is silent abstention rather than wrong extraction; λ isolates this directly.

Over-extraction rate ω (diagnostic). Fraction of negative-label instances on which the model returns a non-empty span. Complements λ: together they decompose the false-negative and false-positive error budgets without conflating them into a single F-score.

All metrics are reported pooled and per prevalence tier.

3.5 Hypotheses

H1. $\exists\, C \in \{\text{M0, M1, M6}\}$ such that C outperforms $B^\star \in \{\text{B1, B4}\}$ on F_2. Null: no such C exists. Decision rule: reject the null if any of the multi-agent vs. baseline pairwise McNemar exact tests (m=33: 3 multi-agent $\times$ 2 baselines $\times$ 11 models, retaining the stronger baseline per model) is significant at α=0.05 after BH correction [BH95], with directional ΔF_2 favouring the multi-agent configuration.

H2. $\Delta\text{Gap}(C_{\text{multi}}, B^\star) < 0$ where $\text{Gap}(C) = F_2^{\text{common}}(C) - F_2^{\text{rare}}(C)$. Null: $\Delta\text{Gap} \geq 0$. Decision rule: reject the null if the 95% paired bootstrap CI on ΔGap excludes zero on the negative side.

H3. M1 and M6 are statistically distinguishable on F_2. Null: $\Delta F_2(\text{M1, M6}) = 0$. Decision rule: reject the null (two-sided) if the McNemar exact p-value is below α=0.05 after BH correction (m=11, one M1 vs. M6 contrast per model). The hypothesis is two-sided because direction is not pre-specified; if rejected, the direction of the effect is reported separately because direction may be inconsistent across the model panel.

3.6 Statistical Methodology

McNemar's exact test. For configurations A and B evaluated on the same instances, let $b = |\{i : A_i \text{ correct}, B_i \text{ wrong}\}|$ and $c = |\{i : A_i \text{ wrong}, B_i \text{ correct}\}|$. The two-sided exact p-value is:

$$p = 2\,\mathbb{P}\big[\text{Bin}\big(b{+}c,\, \tfrac{1}{2}\big) \leq \min(b, c)\big] \tag{3.1}$$

implemented via `scipy.stats.binomtest` [Vir+20]. The exact form is preferred over the χ^2 approximation because discordant counts are small on rare-tier subsets [McN47].

Paired bootstrap. For continuous metrics (F_2, F_1, Jaccard, gap differences), Algorithm 1 gives the paired bootstrap CI [Efr79; RM05]. We use B=300 resamples; the percentile method's robustness to small B on bounded statistics is documented as a sensitivity threat in Section 3.7.

Algorithm 1 Paired bootstrap confidence interval for ΔF_2

Require: Predictions $\hat{y}^A$, $\hat{y}^B$, labels y, B=300, α
1: **for** $b = 1$ **to** B **do**
2: $\mathcal{I} \leftarrow$ sample n indices with replacement from $\{1, \ldots, n\}$
3: $\delta_b \leftarrow F_2(\hat{y}^A_{\mathcal{I}}, y_{\mathcal{I}}) - F_2(\hat{y}^B_{\mathcal{I}}, y_{\mathcal{I}})$
4: **end for**
5: **return** $[\text{percentile}(\delta, \alpha/2),\ \text{percentile}(\delta, 1-\alpha/2)]$

Multiple-comparison correction. Within each hypothesis family, Benjamini–Hochberg [BH95] controls FDR at α=0.05: reject $H_0^{(i)}$ if

$$p_{(i)} \leq \frac{i}{m}\alpha, \quad p_{(1)} \leq \cdots \leq p_{(m)}. \tag{3.2}$$

The H1 family has m=33 tests (3 multi-agent configurations $\times$ 2 baselines $\times$ 11 models, retaining the stronger baseline per model); the H3 family has m=11 tests (one M1 vs. M6 contrast per model). Raw and BH-corrected p-values are both reported.

Effect size. For paired binary comparisons (McNemar's), we report Cohen's $g = b/(b+c) - 0.5$, the deviation of the discordant proportion from chance [Coh88]. For continuous metrics, we report absolute ΔF_2 with the paired bootstrap 95% CI. We follow Cohen's interpretive thresholds ($g \geq 0.05$ small, ≥ 0.15 medium, ≥ 0.25 large; $|\Delta F_2| \geq 0.02$ small, ≥ 0.05 medium, ≥ 0.08 large) [Coh88], and report effect size alongside p-values in accordance with the recommendation that the magnitude of difference is the primary research output, not statistical significance per se [SF12].

Power. Card, Henderson, Khandelwal, et al. [Car+20] caution that NLP comparisons are frequently under-powered. The pairwise sample size after intersection is $\approx$2,900–3,000 per (configuration, model) pair. Under McNemar's exact test at α=0.05 and 80% power, this panel resolves discordant-pair effect sizes corresponding to $|\Delta F_2| \geq 0.025$. Effects below this threshold are treated as inconclusive rather than as evidence of equivalence.

Paired comparison protocol. Intersection of sample IDs is taken symmetrically when partial runs differ in coverage; intersection size is reported with each test.

3.7 Threats to Validity

Internal. Configuration parity is maintained by holding the CUAD loader, sampling seed, prompt registry, and trace schema constant across configurations. M1 and M6 share the same textual specialist instructions (split vs. concatenated), so M1 vs. M6 isolates architecture, not prompt content.

External. Single language (English), single jurisdiction (predominantly U.S. commercial contracts), single corpus (CUAD). Findings on the 81-contract long-document subset may not extrapolate to short contracts, where context-window pressure is lower and the decomposition hypothesis is less applicable. Open-weights models are excluded; inference does not generalise beyond the 11 proprietary models studied. Model snapshots are pinned; provider updates can shift behaviour.

Construct. F_2 is a proxy for legal-review utility, not a direct measurement. Reviewer cost of inspecting a spurious span is workflow-dependent; this is mitigated by reporting precision, recall, λ, and ω separately so that a reviewer with a different cost ratio can re-rank architectures.

Conclusion. Prompt sensitivity is a first-order threat for any architecture comparison in which configurations differ in the number, role, or wording of agent prompts. Chatterjee, Renduchintala, Bhatia, and Chakraborty [Cha+24] report non-trivial sensitivity even on well-defined tasks, with neither parameter count nor instruction tuning systematically reducing it. We mitigate by holding prompt templates fixed across models per configuration, documenting them in full in Chapter 4, and treating prompt-perturbation analysis as future work.

Statistical. Bootstrap percentile CIs under-cover for small samples and skewed statistics [Efr79]; p-values are treated as decisive only when corroborated by the McNemar exact test. The bootstrap resample count (B=300) is below the 1,000 conservative recommendation [Dro+18]; sensitivity at B=1,000 on a representative subset confirmed

CI stability to within ± 0.005 on ΔF_2. Model identity is a fixed factor; inference does not generalise beyond the 11 models studied.

4 System Design and Implementation

This chapter operationalises the five configurations of Chapter 3. The objective is reproducibility: architectures, prompts, state schemas, and trace formats are described at a level of detail sufficient for an independent team to re-implement them from the released artefact. Section 4.1 presents the component diagram and shared data flow. Section 4.2 gives the implementation stack. Section 4.3 defines the run-state schema and prompt registry shared by all configurations. Sections 4.4 and 4.5 specify the per-configuration implementations, including the prompts and routing logic that distinguish M1 from M6. Section 4.6 documents the trace schema underwriting the auditability analysis of Sub-RQ3. Section 4.7 covers the loader, statistical pipeline, and reproducibility commitments. The configuration table is in Chapter 3 (Table 3.1) and is not repeated here.

4.1 System Components and Data Flow

Figure 4.1 shows the static component view. A run is initiated by the experiment driver, which loads CUAD samples through the deterministic loader, resolves prompts through the registry, and dispatches each sample to one of the five LangGraph state machines via a unified model client. Every model call is mirrored to the trace store; final answers are persisted as JSONL records consumed by the statistical pipeline and the inspection dashboard.

4.2 Implementation Stack

The codebase is Python 3.11, managed with `uv`. Orchestration uses LangGraph [Lan24a]; observability uses Langfuse [Lan24b]; the statistical pipeline uses `pandas` [The20], `numpy` [Har+20], and `scipy` [Vir+20]. Provider SDKs are wrapped in a unified `ModelClient` that records, per call, prompt tokens, completion tokens, cache-read tokens (where

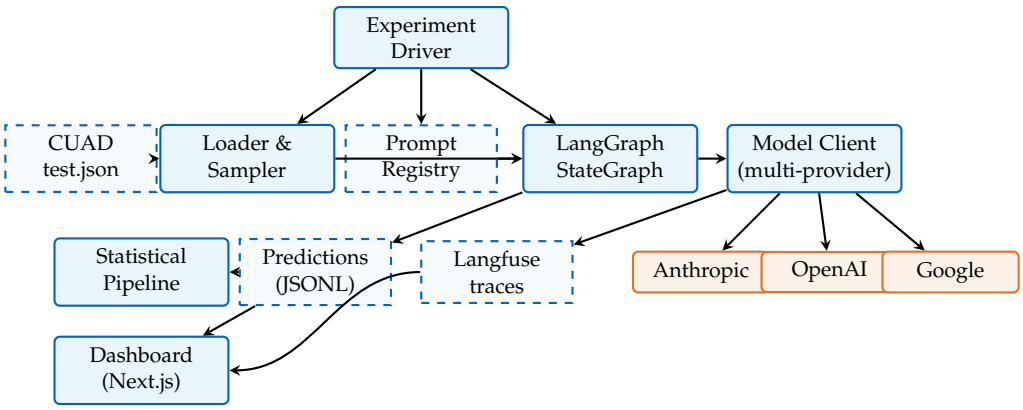


Figure 4.1: System components. Solid-border boxes: thesis code; dashed-border boxes: data stores; orange boxes: external provider APIs. The loader, registry, and client are configuration-agnostic; only the StateGraph topology varies across B1–M6.

supported), wall-clock latency, and provider-billed cost. The wrapper supports 19 model identifiers; only the 11 listed in Table 3.2 are exercised in official runs.

4.3 Shared Abstractions

Graph state. Every configuration is a LangGraph `StateGraph` over a shared `GraphState` object (Listing 4.1). Using a shared schema enforces configuration parity: no configuration has access to fields another lacks, and the JSONL persistence format is identical across all five configurations.

Listing 4.1: GraphState schema (abridged from `src/agents/state.py`).

```
class GraphState(TypedDict, total=False):
    # Inputs (unchanged throughout graph)
    contract_text: str
    category: str # one of 41 CUAD category names
    question: str # CUAD annotation-guideline question text

    # Routing (set by orchestrator node --- M1 only)
    specialist_name: str # "risk_liability" | "temporal_renewal" | "ip_commercial"

    # Extraction (set by specialist or baseline node)
    extraction_result: ExtractionResult | None
```

```
    # Validation (set by validation node)
    validated: bool
    validation_notes: str

    # Final output
    final_result: ExtractionResult | None

    # Trace: append-reducer accumulates entries across all nodes
    trace: Annotated[list[dict[str, Any]], append_trace]

    error: str | None
```

Prompt registry. Prompts are stored as YAML and loaded once per run; the schema version hash is written to the run metadata so prompt drift is detectable post-hoc. Two roots: `prompts/baselines/` (B1, B4, and the M0 classifier and extractor) and `prompts/specialists/` (the three M1 domain specialists and the M6 consolidated prompt). A shared `system/` directory holds anti-laziness instructions and the verbatim-span output contract, referenced by all configurations, ensuring that the behavioural guard-rails are identical across B1, B4, M0, M1, and M6.

Common parser. A single parser converts every model completion to a canonical `(spans: list[str] | None)` representation. For B1, the parser accepts verbatim quoted spans or the literal string "no related clause". For B4 and M0-extractor, the parser additionally requires a `<answer>...</answer>` tag and issues a single repair re-prompt if the tag is absent. On persistent failure the record is marked `parser_status="rejected"` and excluded from official statistics (1,297 of 164,941 predictions, 0.79%).

4.4 Baseline Architectures

B1 — Zero-shot single agent. B1 replicates the zero-shot prompt protocol of [Liu+25]. The model receives one user message containing the contract text, the target category name, the category indicator, and an explicit instruction to return verbatim span(s) or the literal string "no related clause". No examples, no chain-of-thought directives.

B1: zero-shot user template

You are reviewing a commercial contract. Determine whether the contract contains any clause(s) of the category below. If yes, return the verbatim span(s); each span must appear exactly as in the contract. If no, return exactly: "no related clause".
Category: *{{category}}* ***Description:*** *{{category_indicator}}*
Contract: *{{contract_text}}*

B4 — Chain-of-thought single agent. B4 retains B1's task framing but elicits explicit step-by-step reasoning before a final structured answer. The model is instructed to restate what the category covers, scan for candidate spans, weigh ambiguous matches against the indicator, and emit a final `<answer>...</answer>` block. A parsing bug in pilot work caused contract passages quoted inside the chain-of-thought to be harvested as answers, inflating ω (Eq. 2.5); the fix enforces the answer tag via the common parser of Section 4.3. Only post-fix runs enter official statistics.

4.5 Multi-Agent Architectures

M0 — Classify-then-extract. M0 decomposes each (contract, category) task into two sequential LLM calls. The *classifier* returns a single yes/no token; the *extractor* is invoked only if the classifier returns yes. The two agents share the category indicator but have disjoint prompts: the classifier is forbidden to emit spans, the extractor assumes presence. Listing 4.2 excerpts both prompts; Listing 4.3 shows the LangGraph construction.

Listing 4.2: M0 classifier and extractor prompts (abridged).

```
# --- classifier ---
Decide whether the contract contains any clause of the category below.
Return exactly one token: "yes" or "no". Do not return spans.
Category: {{category}} Description: {{category_indicator}}
Contract: {{contract_text}}

# --- extractor (invoked only when classifier returned "yes") ---
The contract contains at least one clause of the category below.
Return the verbatim span(s) wrapped in <answer>...</answer>.
```

Listing 4.3: M0 LangGraph construction.

```
g = StateGraph(GraphState)
g.add_node("classify", classify_node)
```

```
g.add_node("extract", extract_node)
g.set_entry_point("classify")
g.add_conditional_edges(
    "classify",
    lambda s: "extract" if s["classifier_verdict"] == "yes" else END,
    {"extract": "extract", END: END},
)
g.add_edge("extract", END)
```

M1 — Orchestrator with three specialists. M1 routes each request to one of three domain specialists via an LLM router. The category-to-specialist assignment is documented in Table 4.1; it is fixed before any model is run and is committed as `CATEGORY_ROUTING` in `src/agents/orchestrator.py`.

Risk/Liability (13)	**Temporal/Renewal (11)**	**IP/Commercial (17)**
Cap on Liability	Document Name	IP Ownership Assignment
Uncapped Liability	Parties	Joint IP Ownership
Liquidated Damages	Agreement Date	License Grant
Insurance	Effective Date	Non-Transferable License
Warranty Duration	Expiration Date	Affiliate License-Licensor
Audit Rights	Renewal Term	Affiliate License-Licensee
Non-Disparagement	Notice Period to Terminate Renewal	Unlimited License
Covenant Not to Sue	Termination for Convenience	Irrevocable or Perpetual License
Third Party Beneficiary	Anti-Assignment	Source Code Escrow
Most Favored Nation	ROFR/ROFO/ROFN	Exclusivity
Change of Control	Governing Law	Non-Compete
Post-Termination Services		No-Solicit of Customers
Minimum Commitment		No-Solicit of Employees
		Competitive Restriction Exception
		Revenue/Profit Sharing
		Price Restrictions
		Volume Restriction

Table 4.1: Category-to-specialist assignment used by M1. All 41 CUAD categories are listed; the mapping is fixed before any model is run.

The router system prompt is reproduced verbatim below from `ROUTING_SYSTEM_PROMPT` in the artefact. Routing accuracy across the 11-model panel is reported in Section 5.4.

M1 router system prompt (verbatim from `src/agents/orchestrator.py`)

You are a routing agent for a contract analysis system. Your job is to read a question about a contract clause and decide which specialist should handle it.
risk_liability *— Expert in risk allocation, liability, and protective provisions. Covers: liability caps and uncapped liability, liquidated damages, insurance requirements, warranty duration, audit rights, non-disparagement, covenants not to sue, third-party beneficiaries, most favored nation clauses, change of control provisions, post-termination services, and minimum commitments.*
temporal_renewal *— Expert in temporal aspects, document identification, and contract lifecycle. Covers: document name and party identification, agreement/effective/expiration dates, renewal terms and notice periods for renewal termination, termination for convenience, anti-assignment provisions, rights of first refusal/offer/negotiation, and governing law.*
ip_commercial *— Expert in intellectual property rights and commercial restrictions. Covers: IP ownership and assignment, joint IP ownership, license grants (transferability, affiliate licenses, unlimited/perpetual licenses), source code escrow, exclusivity, non-compete and non-solicitation clauses, competitive restriction exceptions, revenue/profit sharing, price restrictions, and volume restrictions.*
Respond with ONLY a JSON object: {"specialist": "<name>", "reasoning": "<brief explanation>"}

Each specialist's prompt is a focused reading instruction for its cluster. The Risk/Liability specialist is shown below; Temporal and IP/Commercial follow the same template.

M1 Risk/Liability specialist system prompt (abridged)

You are a contract analyst specialising in risk-allocation provisions: liability caps, uncapped exclusions, indemnification, insurance, warranty, audit rights, non-disparagement, third-party beneficiaries, most-favored-nation, change of control, post-termination services, and minimum commitments. For these categories, recall of monetary thresholds and explicit carve-outs ("except for...", "not in excess of...") is critical and preferred over summary paraphrase.
Return the verbatim span(s) of any clause of the category below, or "no related clause" if none is present.
Category: *{{category}}* ***Description:*** *{{category_indicator}}*
Contract: *{{contract_text}}*

Figure 4.2 traces a single M1 invocation, showing the parent–child call structure reproduced in the Langfuse trace.

M6 — Consolidated specialist (architectural ablation). M6 is the architectural ablation control. Its prompt is the textual union of the three M1 specialist prompts, concatenated into a single agent with no router, no orchestrator, and no per-category branching:

$$\texttt{prompt}_{\text{M6}} = \text{join}\Big(\texttt{prompt}_{\text{risk}},\ \texttt{prompt}_{\text{temporal}},\ \texttt{prompt}_{\text{ip}}\Big)$$

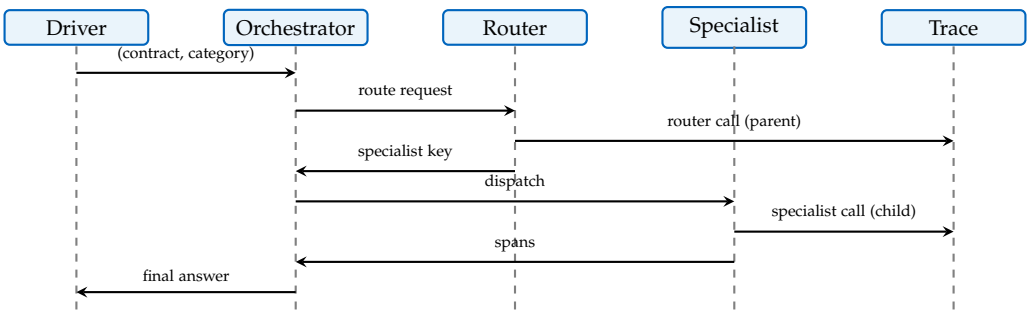


Figure 4.2: M1 sequence diagram for a single (contract, category) invocation. The router call is the Langfuse parent; the specialist call is its child.

The three preambles are separated by domain headings; the verbatim-span output contract is stated once at the end. M6's purpose is methodological: M1 vs. M6 isolates routing and decomposition; M6 vs. B1 isolates prompt content. Together the two contrasts decompose the M1-vs-B1 delta into an architectural component and a prompt-content component, which is the basis for the routing-redundancy finding in Section 5.4.

4.6 Observability and Trace Schema

What is recorded. For each clause-level decision the system captures: (i) configuration ID and model identifier; (ii) prompt template ID, version, and rendered prompt; (iii) full model completion; (iv) parsed final answer with parser version and status; (v) routing decision (M1) or classifier verdict (M0) where applicable; (vi) per-call token counts, billed cost, and wall-clock latency. M1 traces are parent–child structured: the router call is the parent, the specialist call is its child. Listing 4.4 shows a representative M1 record.

Listing 4.4: Representative M1 trace record (abridged JSONL).

```
{
  "contract_id": "CUAD_0231",
  "category": "Cap On Liability",
  "config_id": "M1",
  "model_id": "claude-sonnet-4-6-20251001",
  "calls": [
    { "role": "router",
      "prompt_id": "m1_router/v3",
      "completion": "{\"specialist\":\"risk_liability\",\"reasoning\":\"...\"}",
      "tokens_in": 412, "tokens_out": 18,
```

```
      "latency_ms": 380, "cost_usd": 0.0008,
      "parent_id": null },
    { "role": "specialist_risk_liability",
      "prompt_id": "m1_spec_risk/v2",
      "completion": "<answer>The aggregate liability ... shall not exceed ...</answer>",
      "tokens_in": 14820, "tokens_out": 96,
      "latency_ms": 4210, "cost_usd": 0.0470,
      "parent_id": "router" }
  ],
  "final_answer": "The aggregate liability ... shall not exceed ...",
  "parser_status": "ok"
}
```


Auditability properties. The trace structure supports three concrete properties. First, *decision attribution*: any final answer is traceable to the specific agent, prompt version, and model that produced it. Second, *counterfactual replay*: because prompts and model identifiers are pinned per call, a reviewer can re-run a single call under a different model without re-running the pipeline. Third, *rejection forensics*: when M0 silently returns "no related clause", the classifier's verdict is recorded explicitly, making the classification-gate cascade of Section 5.5.1 visible at audit time as an upstream rejection rather than an unobservable absence.

Trade-offs and limits. The JSONL persistence layer is on the order of gigabytes per full sweep, and trace fields containing rendered prompts and full completions inherit the contract's confidentiality classification. The system distinguishes between decision-level fields (configuration, verdict, token counts) safe to surface to operators, and content-level fields (rendered prompts, full completions) that are access-controlled. The trace is faithful to the call graph, not to the model's internal computation: a chain-of-thought trace records what the model emitted, not what it computed.

Trace completeness. Trace completeness — the fraction of model calls for which all expected fields are populated — was 95.2% for M1 across official runs. The remaining 4.8% are provider-side truncations on long contracts and are flagged in the run logs.

4.7 Experimental Infrastructure

Loader and sampler. The CUAD loader is the single source of truth for which contracts and categories enter a run. It applies the 100,000-character filter, the `neg_ratio=0.7`

natural-prior sampler of Section 3.2, and the prevalence-tier partition. It is deterministic under `seed=42` and produces the identical $\approx$ 2,999-sample envelope across all five configurations, making paired tests in Chapter 5 well-defined at the (contract, category) level.

Statistical aggregation. A separate aggregation pipeline reads per-run JSONL outputs, computes per-sample correctness vectors, and emits the McNemar contingency tables, paired bootstrap distributions, and BH-corrected p-value tables of Section 3.6. Aggregation is deterministic given the JSONL inputs and a fixed bootstrap seed; intermediate artefacts are persisted so results can be regenerated without rerunning models.

Dashboard. A Next.js dashboard supports interactive exploration: per-model F_2 comparisons, per-tier breakdowns, per-call cost and latency views, and trace inspection by sample ID. The over-extraction failure mode in Chapter 5 was first surfaced visually through the dashboard's per-category ω panel.

Test suite and repository layout. An 81-test suite covers the metric implementations (F_1/F_2, Jaccard, λ, ω), the loader (filter, sampling determinism, tier assignment), the agent topologies (each of B1, B4, M0, M1, M6 has at least one end-to-end test against a stub model), and the statistical pipelines (McNemar, bootstrap, BH correction). The full suite passes prior to every official run. The repository is organised as in Listing 4.5.

Listing 4.5: Repository layout (abridged).

```
agentic-ai-for-clm/
  src/
    agents/ # GraphState, orchestrator, 3 specialists, validation
    baselines/ # zero_shot (B1), chain_of_thought (B4), combined (M6)
    data/ # CUAD loader, sampler, tier partition
    evaluation/ # F1/F2/Jaccard/laziness/over-extraction
    models/ # ModelClient, diagnostics, config registry (19 models)
    prompts/ # PromptRegistry + YAML loader
  prompts/
    baselines/ # zero_shot.yaml, chain_of_thought.yaml
    specialists/ # risk_liability.yaml, temporal_renewal.yaml, ip_commercial.yaml
    system/ # extraction_base.yaml (shared anti-laziness instructions)
  configs/experiments/
    baselines.yaml # B1, B4 parameters and calibration targets
    multiagent.yaml # M1, M6 parameters and success criteria
  experiments/results/ # per-run summary JSON + intermediate JSONL
  dashboard/ # Next.js inspection UI (TypeScript, Recharts)
```

```
tests/ # 81 unit + integration tests
notebooks/
  01_data_exploration.ipynb
  02_workflow_test.ipynb
  03_baseline_calibration.ipynb # B1, B4 only
  04_multiagent_experiment.ipynb # M1, M6 + statistical comparison
```

Reproducibility. Three commitments support reproducibility: deterministic sampling under `seed=42`; explicit version pinning of model identifiers (full provider snapshot strings) in the run metadata; and persistence of the full trace bundle (rendered prompts, completions, parsed answers, diagnostics) so that the statistical pipelines of Section 3.6 can be regenerated from raw logs without rerunning models. The prompt registry version hash is written to every run record, making prompt content auditable independently of the run logs.

5 Results

5.1 Run Coverage and Sample Accounting

Table 5.1 reports the per-run sample count for all 55 (model, configuration) cells. Two runs were partial: Claude Sonnet 4 on B1 (n=2,640) and GPT-4.1 on B1 (n=2,528), attributable to upstream sampling interruptions. Every statistical test uses the paired intersection of sample IDs for the specific configuration pair being compared; intersection sizes are reported individually. All other cells reach the target n=2,999.

Model	**B1**	**B4**	**M0**	**M1**	**M6**	**Total**	**B1 cost ($)**
Claude Haiku 4.5	2,999	2,999	2,999	2,999	2,999	14,995	36.70
Claude Opus 4.6	2,999	2,999	2,999	2,999	2,999	14,995	185.02
Claude Sonnet 4	2,640*	2,999	2,999	2,999	2,999	14,636	94.04
Claude Sonnet 4.6	2,999	2,999	2,999	2,999	2,999	14,995	111.44
Gemini 2.5 Flash	2,999	2,999	2,998	2,999	2,999	14,994	10.64
Gemini 2.5 Pro	2,999	2,999	2,999	2,999	2,999	14,995	43.39
Gemini 3 Flash	2,997	2,985	2,984	2,999	2,999	14,964	16.55
Gemini 3.1 Pro	2,994	2,976	2,978	2,999	2,998	14,945	65.75
GPT-4.1	2,528*	2,999	2,786	2,999	2,978	14,290	38.07
GPT-4.1 Mini	2,999	2,994	2,887	2,999	2,987	14,866	13.17
GPT-4.1 Nano	2,998	2,998	2,999	2,999	2,975	14,969	3.17
Column total	32,151	32,946	32,627	32,989	32,931	**163,644**	

Table 5.1: Coverage matrix: per-run sample count and B1 provider cost. *Partial run (upstream sampling interruption); paired tests use the intersection of sample IDs.

Of 164,941 total JSONL records across all 55 runs, 163,644 (99.21%) were analysed after dropping 1,297 records (0.79%) with malformed classification tokens, concentrated in B1 (838 drops) and M0 (362 drops). The natural positive rate across the retained set is 30.1%, confirming the sampler achieved its target class prior.

5.2 H1 — Multi-Agent vs. Single-Agent Baselines

Table 5.2 reports the per-model F_2 for all five configurations. Pooled means are: $\overline{F_2}(\text{B1}) = 0.801$, $\overline{F_2}(\text{B4}) = 0.796$, $\overline{F_2}(\text{M0}) = 0.794$, $\overline{F_2}(\text{M1}) = 0.790$, $\overline{F_2}(\text{M6}) = 0.790$, a range of 0.011 points. No configuration dominates uniformly.

Model	F_2(B1)	F_2(B4)	F_2(M0)	F_2(M1)	F_2(M6)
Claude Haiku 4.5	0.783	**0.849**	0.792	0.835	0.822
Claude Opus 4.6	0.863	**0.865**	0.864	0.860	0.846
Claude Sonnet 4	0.851	0.851	**0.877**	0.826	0.842
Claude Sonnet 4.6	0.813	0.840	**0.860**	0.834	0.836
Gemini 2.5 Flash	**0.829**	0.818	0.619	0.810	0.832
Gemini 2.5 Pro	0.828	0.810	0.707	**0.834**	0.816
Gemini 3 Flash	0.786	0.580	**0.822**	0.747	0.609
Gemini 3.1 Pro	**0.861**	0.843	0.843	0.840	0.854
GPT-4.1	**0.868**	0.852	0.828	0.777	0.853
GPT-4.1 Mini	0.837	0.841	**0.862**	0.777	0.833
GPT-4.1 Nano	0.488	0.612	**0.659**	0.547	0.545
Mean	0.801	0.796	0.794	0.790	0.790

Table 5.2: F_2 by model and configuration. Bold: best configuration per model. Configurations are ordered by pooled mean F_2. Pooled means span a 0.011-point band.

Table 5.3 reports McNemar exact p-values and Benjamini–Hochberg results for the 33 comparisons (M0 vs. B1, M1 vs. B1, M6 vs. B1 per model) constituting the H1 family, together with paired-bootstrap ΔF_2 and 95% CIs.

Figure 5.1 summarises the family-level interaction by showing mean F_2 per (provider family, configuration) cell.

The family interaction is pronounced: the Anthropic family achieves its peak under the B4 chain-of-thought baseline (mean $F_2 = 0.851$), the Google family under B1 (0.826), and the OpenAI family under M0 (0.783). All three Gemini 2.5 runs with M0 show cascading false negatives (Section 5.5.1), which suppresses the Google M0 cell to 0.748.

H1 verdict. The strict null — no multi-agent configuration outperforms the strongest single-agent baseline on F_2 pooled across all eleven models — is not rejected. The 0.011-point range across all five configurations is within sampling noise for any single model. However, 10 of 11 BH-corrected McNemar tests for M0 vs. B1 are significant, and family-conditional effects are real and large (e.g. Gemini 2.5 Flash drops 0.210 F_2

Model	ΔF_2(M0−B1)	CI_{95}	BH	ΔF_2(M1−B1)	CI_{95}	BH
Claude Haiku 4.5	+0.009	[+.050, +.068]	✓	+0.052	[+.060, +.077]	✓
Claude Opus 4.6	+0.001	[+.022, +.036]	✓	−0.004	[+.018, +.031]	✓
Claude Sonnet 4	+0.025	[+.057, +.075]	✓	−0.025	[+.024, +.043]	✓
Claude Sonnet 4.6	+0.047	[+.057, +.072]	✓	+0.021	[+.038, +.055]	✓
Gemini 2.5 Flash	−0.210	[−.045, −.015]	✓	−0.020	[+.021, +.038]	✓
Gemini 2.5 Pro	−0.121	[+.011, +.035]	✓	+0.006	[+.019, +.034]	✓
Gemini 3 Flash	+0.037	[+.043, +.063]	✓	−0.039	[+.010, +.032]	✓
Gemini 3.1 Pro	−0.018	[+.012, +.026]	✓	−0.021	[−.005, +.006]	×
GPT-4.1	−0.040	[+.040, +.061]	✓	−0.091	[+.022, +.041]	✓
GPT-4.1 Mini	+0.025	[+.014, +.027]	×	−0.060	[−.017, −.001]	×
GPT-4.1 Nano	+0.171	[+.067, +.100]	✓	+0.059	[+.024, +.067]	✓

Table 5.3: H1 McNemar exact tests with BH correction (α=0.05, m=33). ΔF_2 is the observed difference from Table 5.2; CI_{95} is the paired-bootstrap interval (B=300). ✓: BH-reject H_0.

points under M0 vs. B1). H1 is therefore conditionally rejected in the negative direction for Gemini models under M0, and conditionally supported for some Anthropic and OpenAI models.

5.3 H2 — Rare-Category Gap

Table 5.4 reports per-tier F_2 for all models under B1, M0, and M1 (the three configurations most relevant to H2). The rare–common gap $g = F_2^{\mathrm{com}} - F_2^{\mathrm{rare}}$ measures how severely each configuration penalises low-prevalence categories.

Under M0, the cascade on Gemini models (Section 5.5.1) collapses F_2^r by up to 0.338 points (Gemini 2.5 Flash: 0.719 → 0.381), inflating the mean gap. Under M1, the rare-tier F_2 degrades severely on the GPT family (GPT-4.1: 0.704 → 0.430; GPT-4.1 Mini: 0.717 → 0.529; GPT-4.1 Nano: 0.253 → 0.167), pointing to specialist over-specialisation on rare categories.

H2 verdict. Rejected. Multi-agent configurations do not reduce the rare–common F_2 gap relative to B1. M0 widens the gap to 0.284 (from 0.276) and M1 widens it to 0.342. The null hypothesis ($\Delta\mathrm{Gap} \geq 0$) cannot be rejected in the favourable direction; in fact, the gap increases significantly under M1, driven by GPT-family specialist failure on rare categories.

Family	B1	B4	M0	M1	M6
Anthropic (4 models)	0.828	0.851	0.848	0.839	0.836
Google (4 models)	0.826	0.763	0.748	0.808	0.777
OpenAI (3 models)	0.731	0.768	0.783	0.700	0.744
All (11 models)	0.801	0.796	0.794	0.790	0.790

Figure 5.1: Mean F_2 per (provider family, configuration) cell. Shading: $\geq$0.84 (dark), 0.80–0.84 (medium), 0.76–0.80 (light), $<$0.76 (very light). Best configuration per family: B4 for Anthropic, B1 for Google, M0 for OpenAI.

5.4 H3 — Architecture vs. Prompts (M1 vs. M6)

Table 5.5 reports the M1–M6 contrasts. The routing accuracy column confirms that all 11 models achieve $\geq$99.03% routing accuracy, with 10 of 11 at 100% (only GPT-4.1 Nano at 99.03%, i.e., 29 mis-routes out of 2,999).

Figure 5.2 shows M1 routing accuracy per model.

Near-perfect routing accuracy is in fact evidence *against* the diagnostic value of routing: the task reduces to a category-name lookup with three disjoint, descriptively-named specialist domains. A deterministic dictionary would achieve the same outcome at zero LLM cost.

Despite identical routing, M1 and M6 are statistically distinguishable on F_2 for all 11 models after BH correction. The direction, however, is inconsistent: M1 outperforms M6 on 5 models (Haiku 4.5, Opus 4.6, Gemini 2.5 Pro, Gemini 3 Flash, GPT-4.1 Nano), and M6 outperforms M1 on 6 models (Sonnet 4, Sonnet 4.6, Gemini 2.5 Flash, Gemini 3.1 Pro, GPT-4.1, GPT-4.1 Mini). The largest absolute differences are Gemini 3 Flash ($+0.138$ for M1) and GPT-4.1 (-0.076 for M6), suggesting a model-capacity interaction: lighter/flash models benefit from the smaller individual specialist prompts, while capable heavy models handle the consolidated prompt more effectively.

H3 verdict. Rejected. M1 and M6 are statistically distinguishable on F_2 for all 11 models ($p < 0.05$ BH-corrected). However, the direction is not consistent across model families. There is no systematic benefit of the routing architecture over the consolidated specialist prompt. The pooled means coincide ($\overline{F_2} = 0.790$ for both), and the per-model differences are explained by a model-capacity interaction rather than by routing contribution.

	B1 (baseline)				M0			M1		
Model	F_2^c	F_2^m	F_2^r	g	F_2^c	F_2^r	g	F_2^c	F_2^r	g
Claude Haiku 4.5	.926	.731	.597	.329	.770	.696	.074	.950	.654	.296
Claude Opus 4.6	.965	.815	.746	.219	.937	.735	.202	.966	.706	.260
Claude Sonnet 4	.958	.791	.714	.245	.940	.730	.211	.948	.691	.257
Claude Sonnet 4.6	.947	.746	.678	.269	.941	.721	.220	.928	.683	.245
Gemini 2.5 Flash	.935	.779	.719	.216	.725	.381	.344	.946	.668	.277
Gemini 2.5 Pro	.958	.768	.690	.268	.919	.403	.515	.969	.711	.258
Gemini 3 Flash	.950	.692	.627	.324	.944	.590	.355	.923	.577	.346
Gemini 3.1 Pro	.966	.807	.741	.225	.962	.672	.289	.963	.701	.262
GPT-4.1	.950	.841	.704	.246	.856	.677	.179	.964	.430	.533
GPT-4.1 Mini	.913	.812	.717	.196	.955	.723	.233	.950	.529	.421
GPT-4.1 Nano	.549	.518	.253	.296	.708	.412	.296	.775	.167	.608
Mean	.929	.773	.653	.276	.878	.594	.284	.940	.574	.342

Table 5.4: Per-tier F_2 for B1, M0, and M1 (c=common, m=moderate, r=rare; columns abbreviated for M0/M1). Gap $g = F_2^c - F_2^r$. Mean gap: B1=0.276, M0=0.284, M1=0.342 — multi-agent configurations widen the rare penalty.

5.5 Additional Findings

5.5.1 The Classification-Gate Rejection Cascade

Table 5.6 quantifies the M0 cascade failure. The M0 classifier emits `predicted_no_clause` (a silent empty return, bypassing the extractor) on a fraction of positive instances. The *gate-caused FN share* measures what fraction of M0 false negatives originate from this path.

The Gemini 2.5 family is the most severe case: wrong-rejection rates of 40.9% and 31.7% versus 2.7% and 1.5% for B1, with $\geq$97.8% of M0 false negatives attributable to the gate. The pattern generalises across all eleven models: on average 82.7% of M0 false negatives are gate-caused. GPT-4.1 Nano is the exception — its B1 laziness (42.0%) exceeds M0's (36.3%), because the model is too weak to reliably extract clauses even without the gate.

The cascade is more severe on rare categories: the Gemini 2.5 Flash rare-tier F_2 drops from 0.719 (B1) to 0.381 (M0), a 0.338-point loss entirely attributable to the gate. This interaction explains why M0 widens the rare–common gap (Section 5.3).

Model	F_2(M1)	F_2(M6)	ΔF_2	CI_{95}	p (McN)	BH
Claude Haiku 4.5	0.835	0.822	+0.013	[+.021, +.035]	1.7×10^{-23}	✓
Claude Opus 4.6	0.860	0.846	+0.014	[+.026, +.038]	7.8×10^{-34}	✓
Claude Sonnet 4	0.826	0.842	−0.016	[−.013, −.001]	2.5×10^{-2}	✓
Claude Sonnet 4.6	0.834	0.836	−0.002	[+.031, +.048]	1.3×10^{-45}	✓
Gemini 2.5 Flash	0.810	0.832	−0.022	[+.015, +.032]	6.8×10^{-21}	✓
Gemini 2.5 Pro	0.834	0.816	+0.018	[+.024, +.038]	2.7×10^{-36}	✓
Gemini 3 Flash	0.747	0.609	+0.138	[+.112, +.140]	1.2×10^{-129}	✓
Gemini 3.1 Pro	0.840	0.854	−0.014	[−.003, +.009]	1.4×10^{-2}	✓
GPT-4.1	0.777	0.853	−0.076	[+.015, +.024]	9.5×10^{-17}	✓
GPT-4.1 Mini	0.777	0.833	−0.056	[+.014, +.032]	5.1×10^{-26}	✓
GPT-4.1 Nano	0.547	0.545	+0.001	[+.011, +.051]	8.2×10^{-15}	✓

Table 5.5: H3: M1 vs. M6 contrasts. $\Delta F_2 = F_2(\mathrm{M1}) - F_2(\mathrm{M6})$ from Table 5.2; CI_{95} from paired bootstrap (B=300); p exact McNemar two-sided; BH at α=0.05, m=33 tests. All 11 models are BH-significant, but direction is inconsistent.

5.5.2 Cost-Efficiency Analysis

Table 5.7 reports per-run provider costs. B1 is the cheapest configuration for all eleven models. The M0 premium over B1 ranges from 1.29× (Gemini 2.5 Flash) to 2.40× (Claude Opus 4.6). M1 and M6 carry smaller premiums (1.00–1.47×), consistent with single additional orchestration calls.

Figure 5.3 plots the F_2–cost Pareto frontier across all 55 (configuration, model) cells. B1 occupies the Pareto frontier at every cost level at which it appears: no configuration is simultaneously cheaper and achieves higher F_2 for the same model. The M0 Gemini cells are dominated in both dimensions (lower F_2 *and* higher cost than B1 Gemini), making them strictly Pareto-inferior.

The cost analysis places B1 on the Pareto frontier: zero configurations are both cheaper and more accurate than B1 for any given model. The 24%–70% premium attached to multi-agent configurations is uncompensated by aggregate F_2 gains. In workflows where the per-agent trace adds operational value (obligation tracking, audit-trail generation), this premium may be acceptable; the Chapter 6 discussion elaborates these conditions.

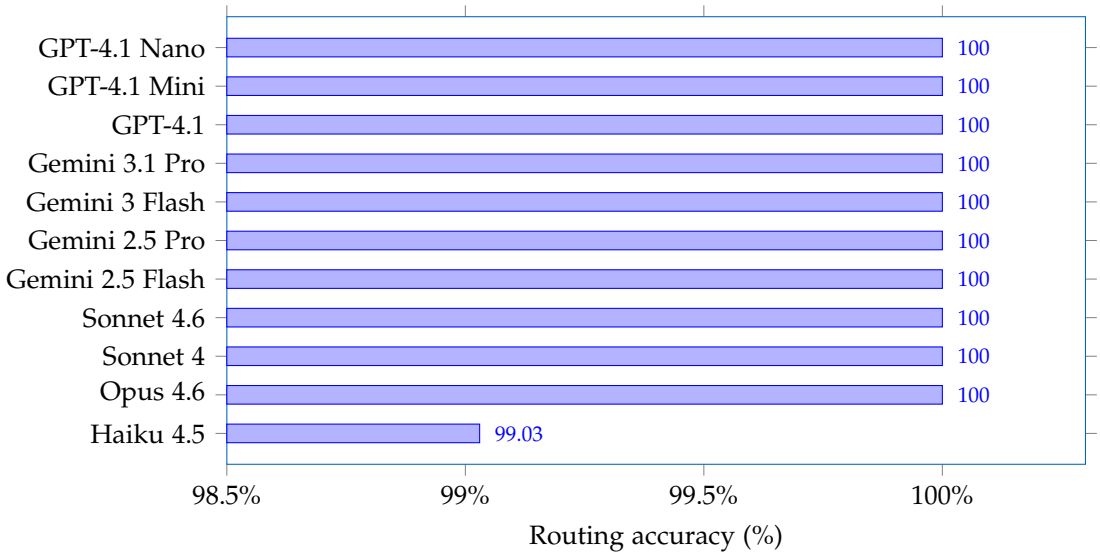


Figure 5.2: M1 router accuracy: agreement with the fixed category-to-specialist mapping. Ten of eleven models achieve 100%; GPT-4.1 Nano reaches 99.03% (29 mis-routes out of 2,999).

Model	M0 laz.	B1 laz.	M0 FN	Gate-FN	Gate %
Claude Haiku 4.5	16.9%	3.9%	255	210	82.4%
Claude Opus 4.6	7.6%	1.9%	142	95	66.9%
Claude Sonnet 4	7.0%	2.1%	123	87	70.7%
Claude Sonnet 4.6	10.5%	2.0%	156	131	84.0%
Gemini 2.5 Flash	40.9%	2.7%	520	509	97.9%
Gemini 2.5 Pro	31.7%	1.5%	403	394	97.8%
Gemini 3 Flash	17.9%	2.0%	221	197	89.1%
Gemini 3.1 Pro	15.6%	3.8%	191	169	88.5%
GPT-4.1	17.8%	5.6%	207	191	92.3%
GPT-4.1 Mini	11.8%	8.4%	147	103	70.1%
GPT-4.1 Nano	36.3%	42.0%	452	318	70.4%
Mean	19.5%	6.5%	256	218	82.7%

Table 5.6: M0 classification-gate rejection cascade. Laz. = laziness rate (FN fraction of positives). Gate-FN = false negatives where `predicted_no_clause` = True. Gate % = Gate-FN / M0 FN $\times$100. Gemini 2.5 Flash and Pro: $\approx$98% of M0 false negatives are gate-caused.

Model	B1 ($)	B4 ($)	M0 ($)	M1 ($)	M6 ($)
Claude Haiku 4.5	36.70	43.11	60.66	45.00	43.29
Claude Opus 4.6	185.02	213.30	302.96	215.49	211.10
Claude Sonnet 4	94.04	125.98	184.58	127.17	125.41
Claude Sonnet 4.6	111.44	129.59	184.11	137.75	128.24
Gemini 2.5 Flash	10.64	14.72	13.73	13.31	13.99
Gemini 2.5 Pro	43.39	61.77	58.69	55.57	58.21
Gemini 3 Flash	16.55	19.09	26.24	19.65	19.18
Gemini 3.1 Pro	65.75	72.46	104.57	78.73	72.69
GPT-4.1	38.07	77.88	91.50	56.11	73.68
GPT-4.1 Mini	13.17	15.78	21.42	13.22	15.78
GPT-4.1 Nano	3.17	3.64	5.10	3.64	3.63
Mean ratio vs. B1	1.00×	1.30×	1.67×	1.23×	1.27×

Table 5.7: Provider cost per run (2,999 samples). Mean ratio: average of per-model cost / B1-cost. M0 carries a 1.67× mean premium; B4 is 1.30×; M1 and M6 are ≈1.25×.

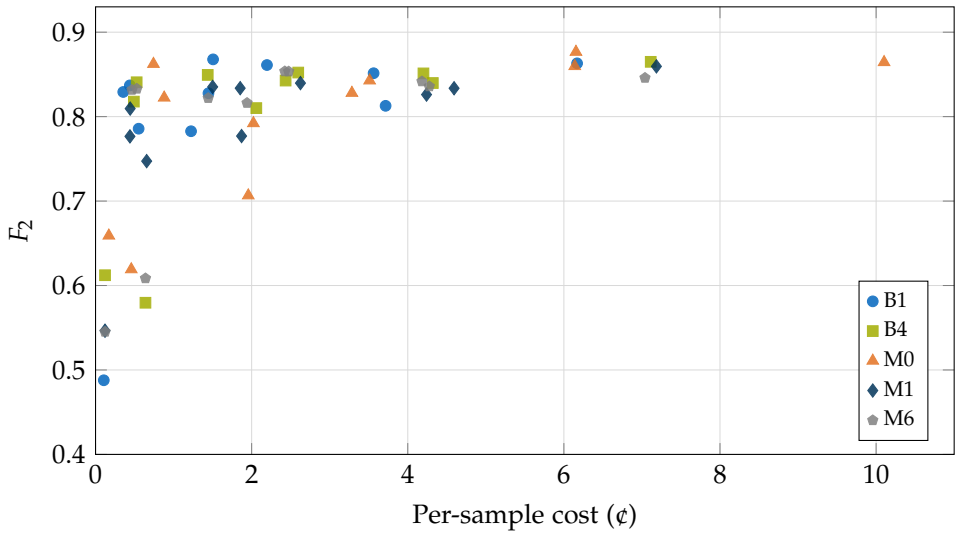


Figure 5.3: F_2 vs. per-sample provider cost for all 55 (configuration, model) cells. B1 (circle) dominates or matches every configuration at every cost level. Three M0 outliers (Gemini 2.5 Flash/Pro and Gemini 3 Flash) are dominated in both dimensions.

6 Discussion

6.1 When Multi-Agent Helps: Evidence for Pipeline Decomposition

6.2 When Multi-Agent Hurts: The LLM-Router Problem

6.3 Architecture vs Prompts: The H3 Verdict

6.4 Practitioner Perspective: Gaps Between Benchmarks and Real-World CLM

6.5 Connecting to Prior Work

6.6 Limitations

7 Conclusion and Outlook

7.1 Summary of Contributions

7.2 Answers to Research Questions

7.3 Outlook and Future Work

List of Figures

2.1 CLM lifecycle. Clause extraction, risk identification, and obligation tracking (highlighted) occur primarily in the Execute and Monitor stages. 6

2.2 Multi-agent design space: five structural axes and their principal options. Shaded boxes mark options covered by at least one experimental configuration (B1, B4, M0, M1, M6) in this thesis. Chapter 3 specifies which configuration selects which option. 8

2.3 Approximate F_1 scores for 13 representative CUAD categories under GPT-4.1 (ContractEval [Liu+25]). Values are indicative; exact scores vary by model. Rare-tier categories achieve near-zero F_1 under the strongest single-agent baseline, directly motivating the per-domain specialist design evaluated in Chapter 3. 10

3.1 Message-flow diagrams for the five configurations. Input = contract text + clause-category indicator. Blue nodes: generalist LLM invocations. Orange nodes: domain-specialist prompts. $\emptyset$ = silent no-clause return. 18

4.1 System components. Solid-border boxes: thesis code; dashed-border boxes: data stores; orange boxes: external provider APIs. The loader, registry, and client are configuration-agnostic; only the StateGraph topology varies across B1–M6. 26

4.2 M1 sequence diagram for a single (contract, category) invocation. The router call is the Langfuse parent; the specialist call is its child. 31

5.1 Mean F_2 per (provider family, configuration) cell. Shading: $\geq$0.84 (dark), 0.80–0.84 (medium), 0.76–0.80 (light), $<$0.76 (very light). Best configuration per family: B4 for Anthropic, B1 for Google, M0 for OpenAI. 38

5.2 M1 router accuracy: agreement with the fixed category-to-specialist mapping. Ten of eleven models achieve 100%; GPT-4.1 Nano reaches 99.03% (29 mis-routes out of 2,999). 41

5.3 F_2 vs. per-sample provider cost for all 55 (configuration, model) cells. B1 (circle) dominates or matches every configuration at every cost level. Three M0 outliers (Gemini 2.5 Flash/Pro and Gemini 3 Flash) are dominated in both dimensions. 42

List of Tables

2.1 Frontier LLM releases referenced in this thesis. Context-window figures denote the standard production context; beta long-context modes noted parenthetically. Capabilities summarised from official announcements. 7
2.2 CUAD dataset statistics [Hen+21]. 9
2.3 Main contract-analysis benchmarks. 11
2.4 Multi-agent legal AI systems coded against the five design-space axes of Section 2.1.3. All prior systems evaluate a single architecture without ablation. 12

3.1 Five evaluated configurations. M0 vs. B1 isolates the classification gate; M1 vs. M6 isolates routing and decomposition; B4 vs. B1 isolates chain-of-thought. 17
3.2 Eleven frontier proprietary models evaluated. Three families; compute-tier range covers light-to-heavy within each family. 19

4.1 Category-to-specialist assignment used by M1. All 41 CUAD categories are listed; the mapping is fixed before any model is run. 29

5.1 Coverage matrix: per-run sample count and B1 provider cost. *Partial run (upstream sampling interruption); paired tests use the intersection of sample IDs. 35
5.2 F_2 by model and configuration. Bold: best configuration per model. Configurations are ordered by pooled mean F_2. Pooled means span a 0.011-point band. 36
5.3 H1 McNemar exact tests with BH correction (α=0.05, m=33). ΔF_2 is the observed difference from Table 5.2; CI_{95} is the paired-bootstrap interval (B=300). ✓: BH-reject H_0. 37
5.4 Per-tier F_2 for B1, M0, and M1 (c=common, m=moderate, r=rare; columns abbreviated for M0/M1). Gap $g = F_2^c - F_2^r$. Mean gap: B1=0.276, M0=0.284, M1=0.342 — multi-agent configurations widen the rare penalty. 39

5.5 H3: M1 vs. M6 contrasts. $\Delta F_2 = F_2(\mathrm{M1}) - F_2(\mathrm{M6})$ from Table 5.2; CI_{95} from paired bootstrap (B=300); p exact McNemar two-sided; BH at α=0.05, m=33 tests. All 11 models are BH-significant, but direction is inconsistent. 40
5.6 M0 classification-gate rejection cascade. Laz. = laziness rate (FN fraction of positives). Gate-FN = false negatives where `predicted_no_clause` = True. Gate % = Gate-FN / M0 FN ×100. Gemini 2.5 Flash and Pro: ≈98% of M0 false negatives are gate-caused. 41
5.7 Provider cost per run (2,999 samples). Mean ratio: average of per-model cost / B1-cost. M0 carries a 1.67× mean premium; B4 is 1.30×; M1 and M6 are ≈1.25×. 42

Bibliography

[Ant24] Anthropic. *Introducing the Model Context Protocol*. Anthropic News Release. Released 25 November 2024. Nov. 2024.

[AO 23] A&O Shearman. *A&O announces exclusive launch partnership with Harvey*. Press Release. Feb. 2023.

[BH95] Y. Benjamini and Y. Hochberg. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing." In: *Journal of the Royal Statistical Society, Series B (Methodological)* 57.1 (1995), pp. 289–300. DOI: 10.1111/j.2517-6161.1995.tb02031.x.

[Bog+25] T. Bogavelli et al. *AgentArch: A Comprehensive Benchmark to Evaluate Agent Architectures in Enterprise*. ServiceNow Research. Full author list at arXiv:2509.10769. 2025. arXiv: 2509.10769.

[Car+20] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky. "With Little Power Comes Great Responsibility." In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*. 2020, pp. 9263–9274. DOI: 10.18653/v1/2020.emnlp-main.745. arXiv: 2010.06595.

[Cem+25] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. *Why Do Multi-Agent LLM Systems Fail?* 2025. arXiv: 2503.13657.

[Cha+20] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos. "LEGAL-BERT: The Muppets Straight Out of Law School." In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. 2020, pp. 2898–2904. arXiv: 2010.02559.

[Cha+24] A. Chatterjee, H. S. V. N. S. K. Renduchintala, S. Bhatia, and T. Chakraborty. "POSIX: A Prompt Sensitivity Index for Large Language Models." In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2024, pp. 14550–14565. arXiv: 2410.02185.

[Coh88] J. Cohen. *Statistical Power Analysis for the Behavioral Sciences*. 2nd. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988. ISBN: 0-8058-0283-5.

[Cui+24] J. Cui, M. Ning, Z. Li, B. Chen, Y. Yan, H. Li, B. Ling, Y. Tian, and L. Yuan. *ChatLaw: A Multi-Agent Collaborative Legal Assistant with Knowledge Graph Enhanced Mixture-of-Experts Large Language Model*. Version 2 (2024); v1 (2023) is the original retrieval-augmented system. 2024. arXiv: 2306.16092.

[Dev+19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019)*. 2019, pp. 4171–4186. arXiv: 1810.04805.

[Dom+24] R. Dominguez-Olmedo, V. Nanda, R. Abebe, S. Bechtold, C. Engel, J. Frankenreiter, K. Gummadi, M. Hardt, and M. Livermore. *Lawma: The Power of Specialization for Legal Annotation*. ICLR 2025. 2024. arXiv: 2407.16615 [cs.CL].

[Dro+18] R. Dror, G. Baumer, S. Shlomov, and R. Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing." In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018)*. 2018, pp. 1383–1392.

[EC16] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 4.5.2016, pp. 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj. 2016.

[EC24] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L 2024/1689, 12.7.2024. http://data.europa.eu/eli/reg/2024/1689/oj. 2024.

[Efr79] B. Efron. "Bootstrap Methods: Another Look at the Jackknife." In: *The Annals of Statistics* 7.1 (1979), pp. 1–26. DOI: 10.1214/aos/1176344552.

[Gao+25] M. Gao, Y. Li, B. Liu, Y. Yu, P. Wang, C.-Y. Lin, and F. Lai. *Single-agent or Multi-agent Systems? Why Not Both?* 2025. arXiv: 2505.18286.

[Har+20] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. "Array Programming with NumPy." In: *Nature* 585 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[Hat+23] J. Hatzius, J. Briggs, D. Kodnani, and G. Pierdomenico. *The Potentially Large Effects of Artificial Intelligence on Economic Growth (Briggs/Kodnani)*. Goldman Sachs Global Economics Analyst. Published 26 March 2023. Primary report is paywalled; widely cited in *The Globe and Mail* and *Law.com*. Mar. 2023.

[Hen+21] D. Hendrycks, C. Burns, A. Chen, and S. Ball. "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review." In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS 2021)*. 2021. arXiv: 2103.06268.

[Hon+24] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework." In: *12th International Conference on Learning Representations (ICLR 2024)*. Oral presentation. 2024. arXiv: 2308.00352.

[KM21] Y. Koreeda and C. D. Manning. "ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts." In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 1907–1919. arXiv: 2110.01799.

[Lam+16] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer. "Neural Architectures for Named Entity Recognition." In: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2016)*. 2016, pp. 260–270. arXiv: 1603.01360.

[Lan24a] LangChain Inc. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. Software library. https://github.com/langchain-ai/langgraph. 2024.

[Lan24b] Langfuse GmbH. *Langfuse: Open-Source LLM Engineering Platform*. Software / documentation. https://langfuse.com/ ; https://github.com/langfuse/langfuse. 2024.

[Li+23] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. "CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2303.17760.

[Liu+25] S. Liu, Z. Li, R. Ma, H. Zhao, and M. Du. *ContractEval: Benchmarking LLMs for Clause-Level Legal Risk Identification in Commercial Contracts*. 2025. arXiv: 2508.03080.

[LMP01] J. D. Lafferty, A. McCallum, and F. C. N. Pereira. "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data." In: *Proceedings of the 18th International Conference on Machine Learning (ICML 2001)*. 2001, pp. 282–289.

[Mar+24] L. Martin, N. Whitehouse, S. Yiu, L. Catterson, and R. Perera. *Better Call GPT: Comparing Large Language Models Against Lawyers*. Onit AI Centre of Excellence. Vendor-funded; directional findings replicated by independent evaluations. 2024. arXiv: 2401.16212.

[McN47] Q. McNemar. "Note on the Sampling Error of the Difference between Correlated Proportions or Percentages." In: *Psychometrika* 12.2 (1947), pp. 153–157. DOI: 10.1007/BF02295996.

[PwC23] PwC. *PwC partners with OpenAI and Harvey to build domain specific foundation models*. PwC Press Release. 2023.

[Rap+25] P. Raptopoulos, G. Filandrianos, M. Lymperaiou, and G. Stamou. "PAKTON: A Multi-Agent Framework for Question Answering in Long Legal Agreements." In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*. Oral presentation. 2025. arXiv: 2506.00608.

[RM05] S. Riezler and J. T. Maxwell III. "On Some Pitfalls in Automatic Evaluation and Significance Testing for MT." In: *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. 2005, pp. 57–64.

[San25] M. Sanwal. *Layered Chain-of-Thought Prompting for Multi-Agent LLM Systems: A Comprehensive Approach to Explainable Large Language Models*. 2025. arXiv: 2501.18645.

[SF12] G. M. Sullivan and R. Feinn. "Using Effect Size—or Why the P Value Is Not Enough." In: *Journal of Graduate Medical Education* 4.3 (2012), pp. 279–282. DOI: 10.4300/JGME-D-12-00156.1.

[Shi+23] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. "Reflexion: Language Agents with Verbal Reinforcement Learning." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2303.11366.

[Sur+25] R. Surapaneni et al. *Announcing the Agent2Agent Protocol (A2A)*. Google Developers Blog. Subsequently donated to the Linux Foundation, 23 June 2025. Apr. 2025.

[The20] The pandas development team. *pandas-dev/pandas: Pandas*. Zenodo. Cite the Zenodo release matching the version used in the study. 2020. DOI: 10.5281/zenodo.3509134.

[Tug+20] D. Tuggener, P. von Däniken, T. Peetz, and M. Cieliebak. "LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts." In: *Proceedings of the 12th Language Resources and Evaluation Conference (LREC 2020)*. 2020, pp. 1235–1241.

[Vir+20] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[Wan+23a] S. H. Wang, A. Scardigli, L. Tang, W. Chen, D. Levkin, A. Chen, S. Ball, T. Woodside, O. Zhang, and D. Hendrycks. "MAUD: An Expert-Annotated Legal NLP Dataset for Merger Agreement Understanding." In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*. 2023, pp. 16369–16382. arXiv: 2301.00876.

[Wan+23b] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." In: *11th International Conference on Learning Representations (ICLR 2023)*. 2023. arXiv: 2203.11171.

[Wan+24] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou. *Mixture-of-Agents Enhances Large Language Model Capabilities*. Also published at ICLR 2025. 2024. arXiv: 2406.04692.

[Wei+22] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." In: *Advances in Neural Information Processing Systems (NeurIPS 2022)*. 2022. arXiv: 2201.11903.

[Wor25] World Commerce & Contracting. *Contract Management: An Overlooked Driver of Business Agility and Financial Performance*. WorldCC Whitepaper. Available at https://www.worldcc.com/Portals/IACCM/Reports/Contract%20Management%20Whitepaper.pdf. 2025.

[Wu+23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework*. Also published at COLM 2024. 2023. arXiv: 2308.08155.

[WY25] Z. Wang and B. L. Yuan. *L-MARS: Legal Multi-Agent Workflow with Orchestrated Reasoning and Agentic Search*. 2025. arXiv: 2509.00761.

[Yao+23a] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2305.10601.

[Yao+23b] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models." In: *11th International Conference on Learning Representations (ICLR 2023)*. 2023. arXiv: 2210.03629.